

CMU 14-455 (2019) · 课程资料包 @ShowMeAI



视频

中英双语字幕



课件

一键打包下载



笔记

官方笔记翻译



代码

作业项目解析



视频 · B 站 [扫码或点击链接]

<https://www.bilibili.com/video/BV1qf4y1J7mX>



课件 & 代码 · 博客 [扫码或点击链接]

<http://blog.showmeai.tech/cmu-14-455/>

高级 SQL

数据库

聚合

内存管理

树索引

排序

查询规划

索引构建

分组

并发控制

查询执行

Awesome AI Courses Notes Cheatsheets 是 [ShowMeAI](#) 资料库的分支系列，覆盖最具知名度的 **TOP50+** 门 AI 课程，旨在为读者和学习者提供一整套高品质中文学习笔记和速查表。

点击课程名称，跳转至课程**资料包**页面，**一键下载**课程全部资料！

机器学习	深度学习	自然语言处理	计算机视觉
Stanford · CS229	Stanford · CS230	Stanford · CS224n	Stanford · CS231n
# Awesome AI Courses Notes Cheatsheets · 持续更新中			
知识图谱	图机器学习	深度强化学习	自动驾驶
Stanford · CS520	Stanford · CS224W	UCBerkeley · CS285	MIT · 6.S094



微信公众号

资料下载方式 2：扫码点击**底部菜单栏**

称为 **AI 内容创作者**？回复 [添砖加瓦]

18

Timestamp Ordering Concurrency Control



Intro to Database Systems
15-445/15-645
Fall 2019

AP

Andy Pavlo
Computer Science
Carnegie Mellon University

CONCURRENCY CONTROL APPROACHES

Two-Phase Locking (2PL)

→ Determine serializability order of conflicting operations at runtime while txns execute.

Pessimistic

Timestamp Ordering (T/O)

→ Determine serializability order of txns before they execute.

Optimistic

T/O CONCURRENCY CONTROL

Use timestamps to determine the serializability order of txns.

If $TS(T_i) < TS(T_j)$, then the DBMS must ensure that the execution schedule is equivalent to a serial schedule where T_i appears before T_j .

TIMESTAMP ALLOCATION

Each txn T_i is assigned a unique fixed timestamp that is monotonically increasing.

- Let $TS(T_i)$ be the timestamp allocated to txn T_i .
- Different schemes assign timestamps at different times during the txn.

Multiple implementation strategies:

- System Clock.
- Logical Counter.
- Hybrid.

TODAY'S AGENDA

Basic Timestamp Ordering Protocol

Optimistic Concurrency Control

Partition-based Timestamp Ordering

Isolation Levels



BASIC T/O

Txns read and write objects without locks.

Every object **X** is tagged with timestamp of the last txn that successfully did read/write:

→ **W-TS(X)** – Write timestamp on **X**

→ **R-TS(X)** – Read timestamp on **X**

Check timestamps for every operation:

→ If txn tries to access an object "from the future", it aborts and restarts.

BASIC T/O – READS

If $TS(T_i) < W-TS(X)$, this violates timestamp order of T_i with regard to the writer of X .

→ Abort T_i and restart it with a **newer** TS.

Else:

→ Allow T_i to read X .

→ Update $R-TS(X)$ to $\max(R-TS(X), TS(T_i))$

→ Have to make a local copy of X to ensure repeatable reads for T_i .

BASIC T/O – WRITES

If $TS(T_i) < R-TS(X)$ or $TS(T_i) < W-TS(X)$

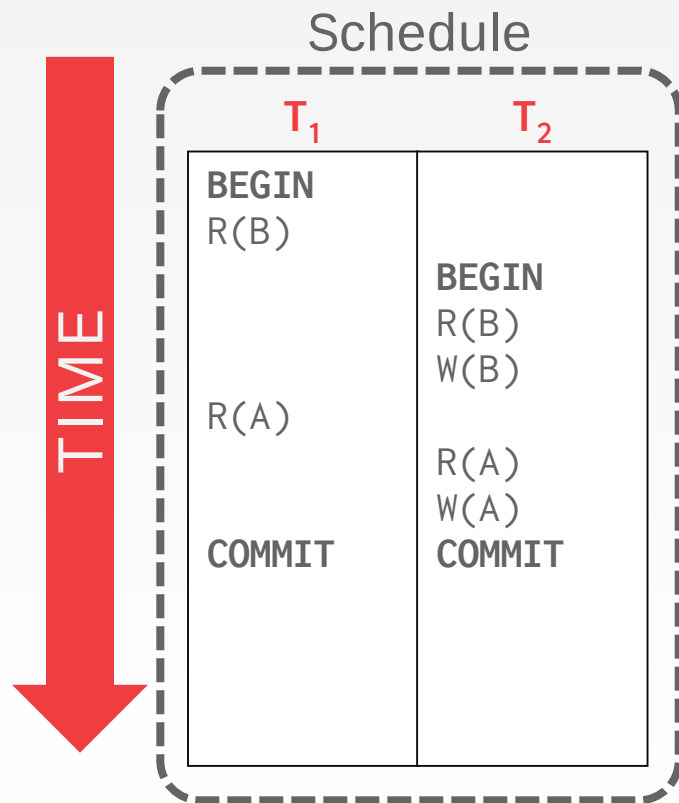
→ Abort and restart T_i .

Else:

→ Allow T_i to write X and update $W-TS(X)$

→ Also have to make a local copy of X to ensure repeatable reads for T_i .

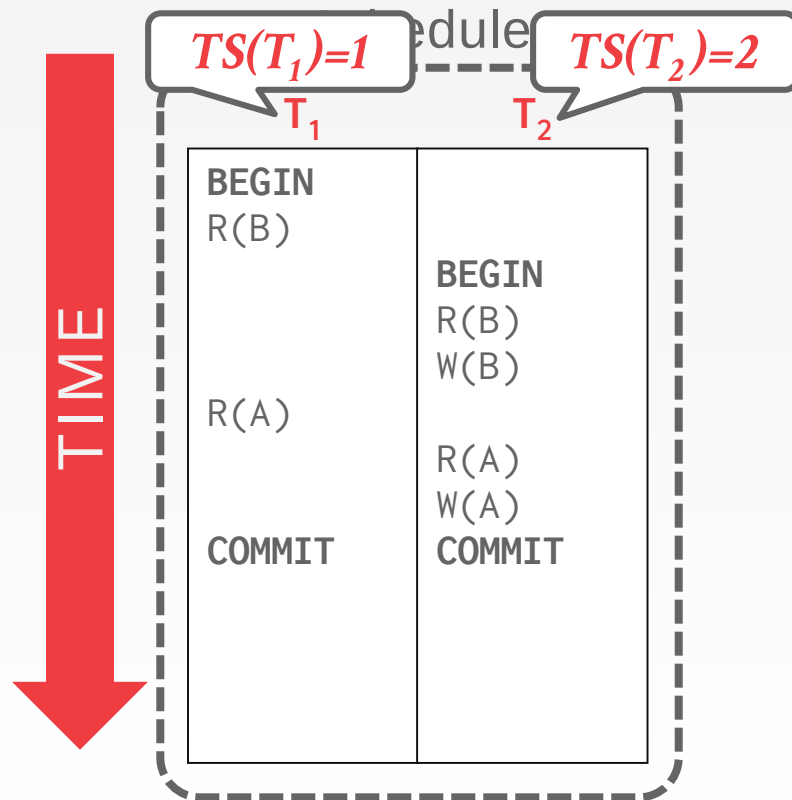
BASIC T/O – EXAMPLE #1



Database

Object	R-TS	W-TS
A	0	0
B	0	0

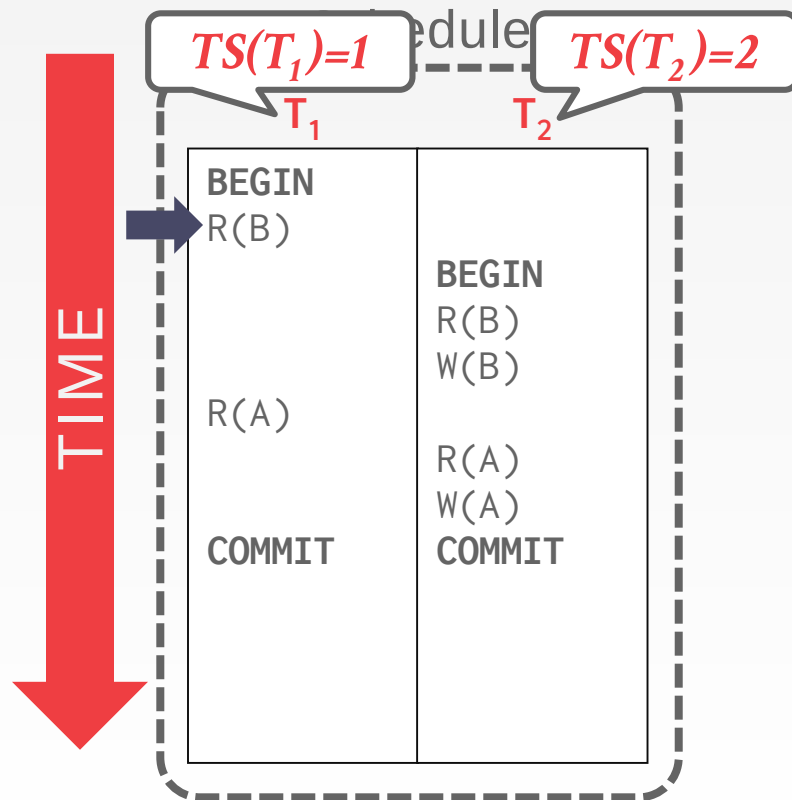
BASIC T/O – EXAMPLE #1



Database

Object	R-TS	W-TS
A	0	0
B	0	0

BASIC T/O – EXAMPLE #1

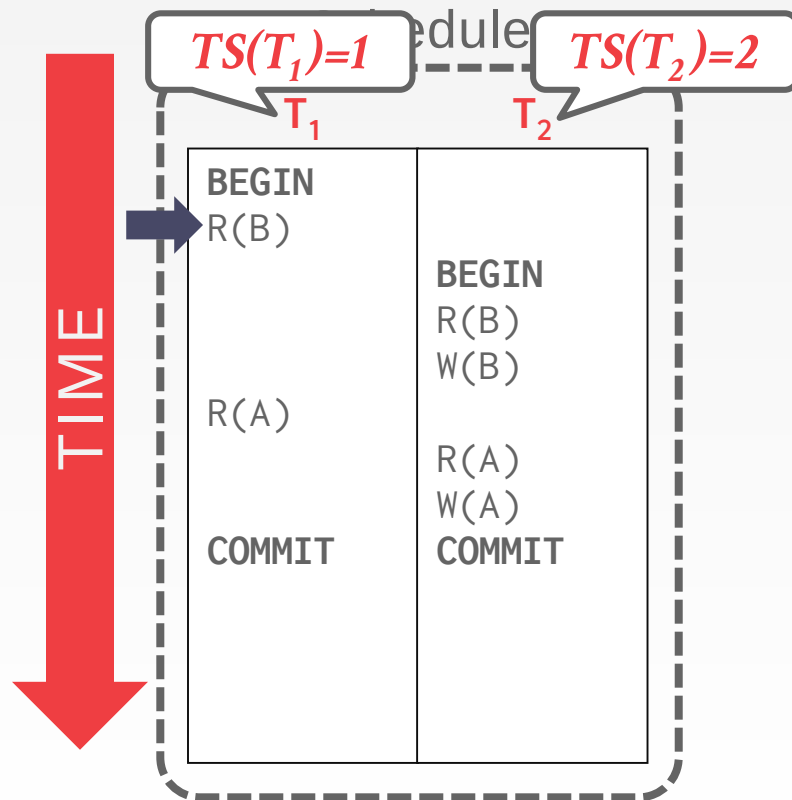


Database

Object	R-TS	W-TS
A	0	0
B	0	0

(Note: The value 0 in the R-TS column for Object B is circled in red in the original image.)

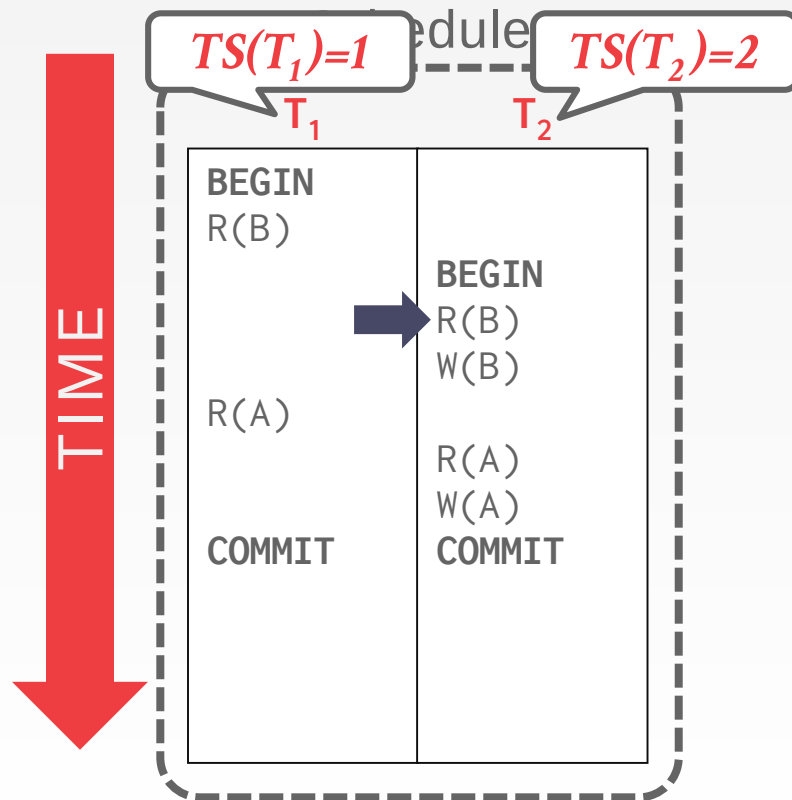
BASIC T/O – EXAMPLE #1



Database

Object	R-TS	W-TS
A	0	0
B	1	0

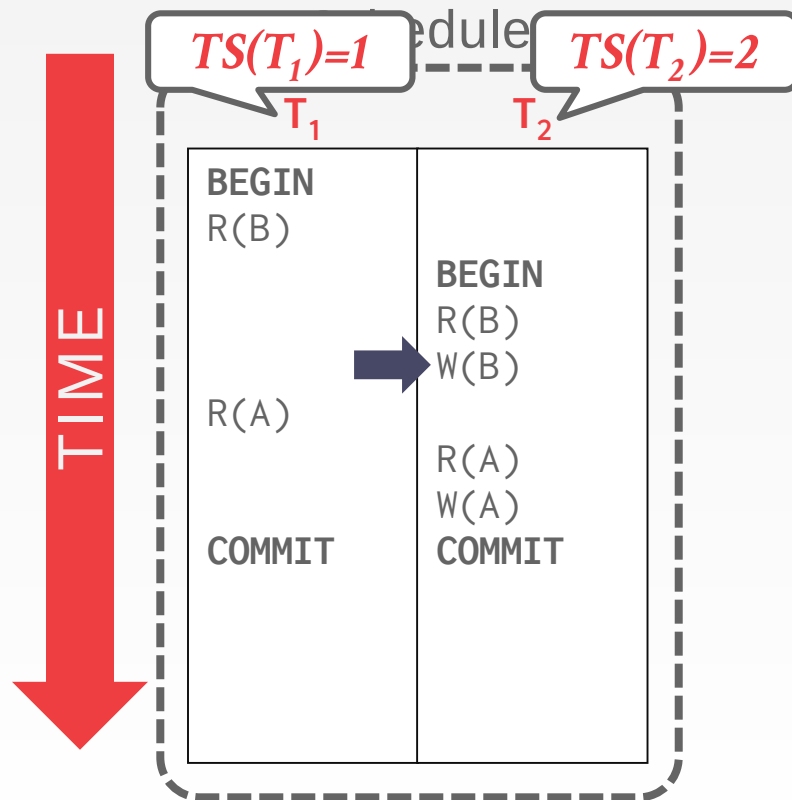
BASIC T/O – EXAMPLE #1



Database

Object	R-TS	W-TS
A	0	0
B	2	0

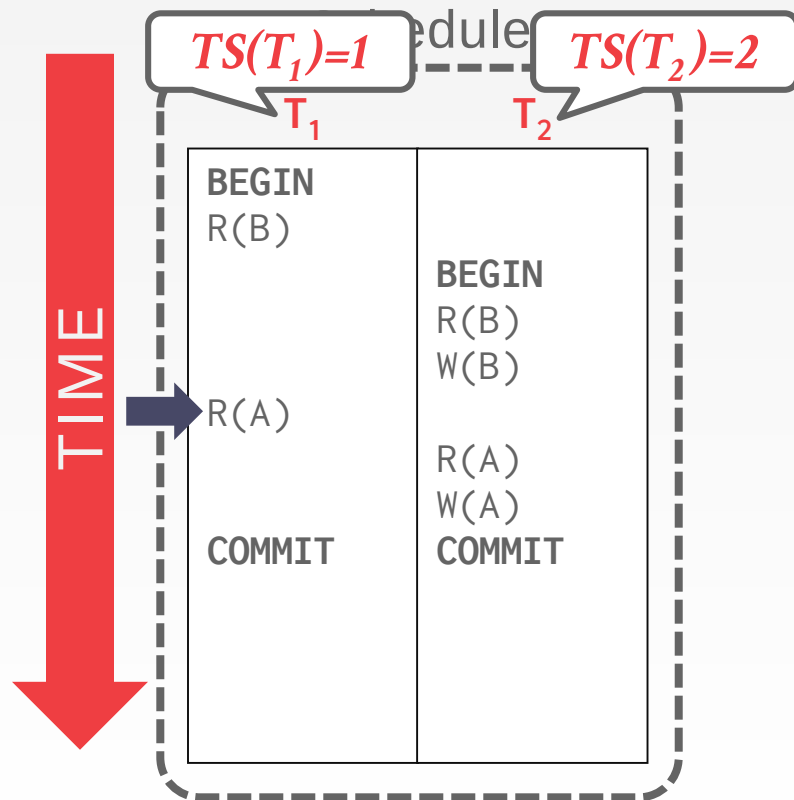
BASIC T/O – EXAMPLE #1



Database

Object	R-TS	W-TS
A	0	0
B	2	2

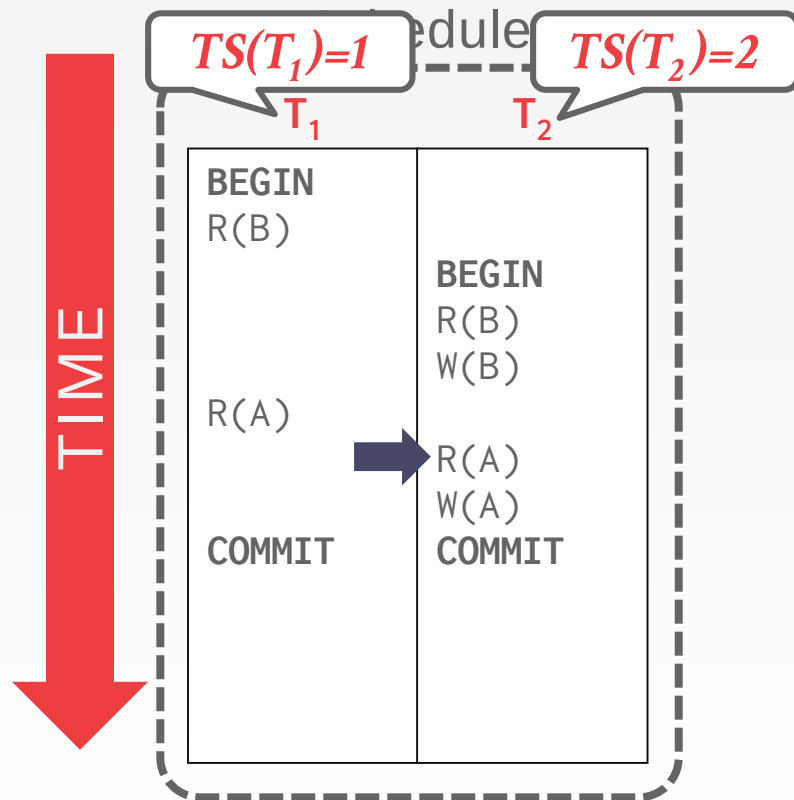
BASIC T/O – EXAMPLE #1



Database

Object	R-TS	W-TS
A	1	0
B	2	2

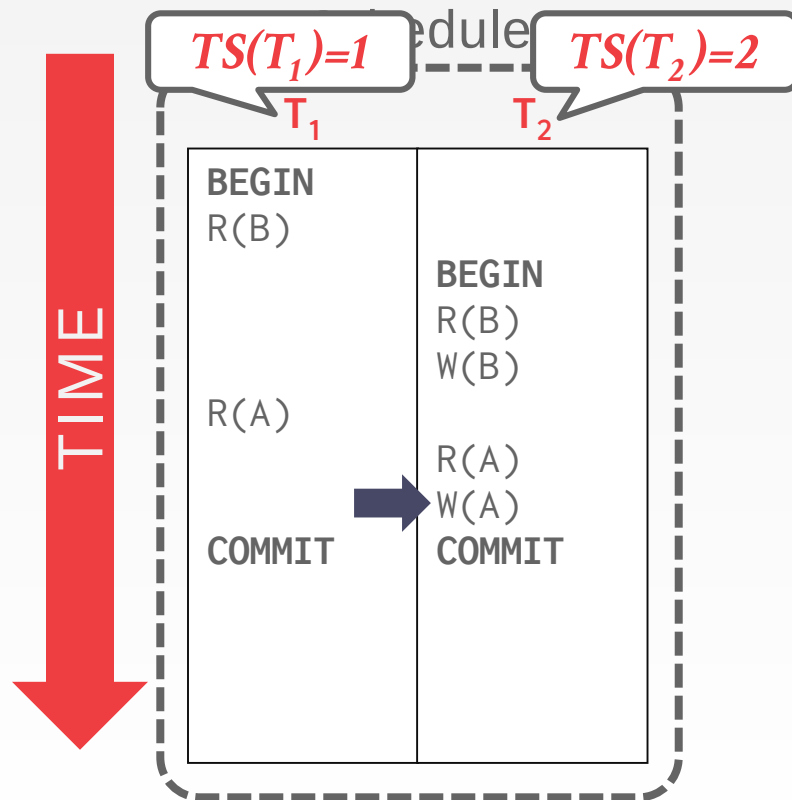
BASIC T/O – EXAMPLE #1



Database

Object	R-TS	W-TS
A	2	0
B	2	2

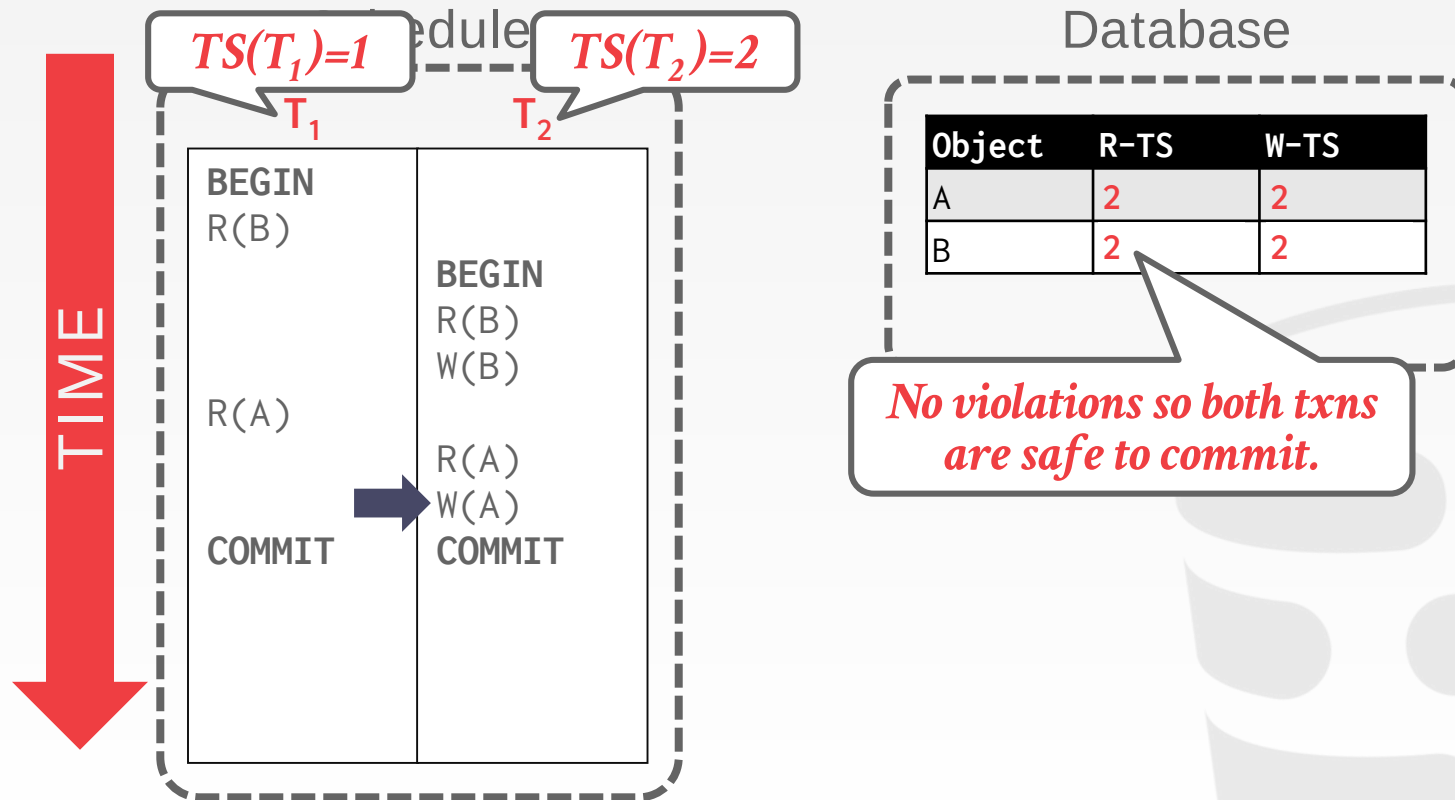
BASIC T/O – EXAMPLE #1



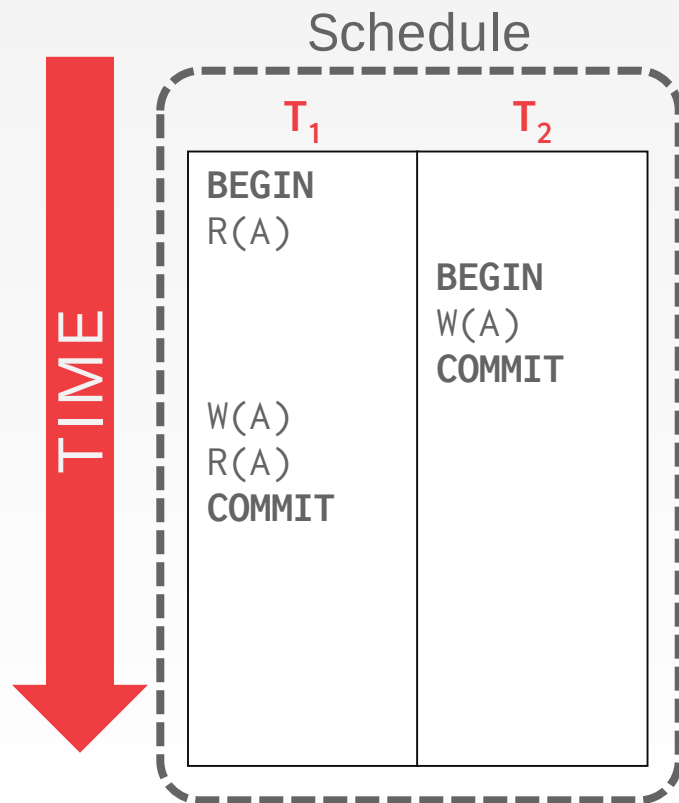
Database

Object	R-TS	W-TS
A	2	2
B	2	2

BASIC T/O – EXAMPLE #1



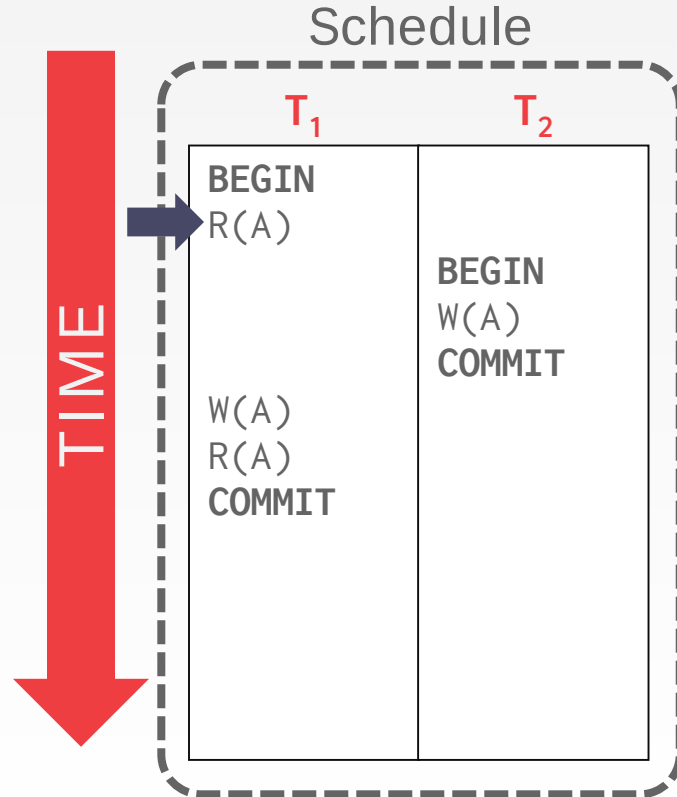
BASIC T/O – EXAMPLE #2



Database

Object	R-TS	W-TS
A	0	0
B	0	0

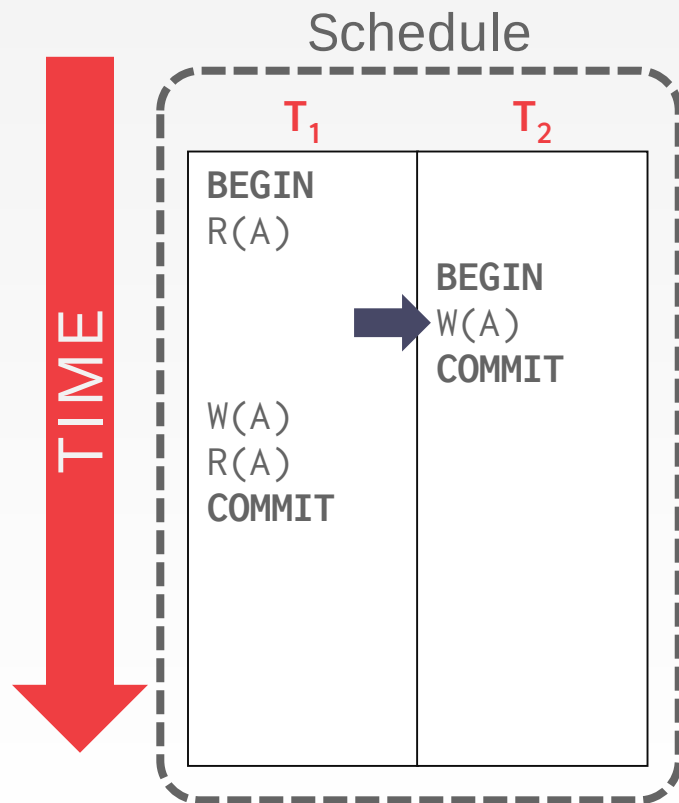
BASIC T/O – EXAMPLE #2



Database

Object	R-TS	W-TS
A	1	0
B	0	0

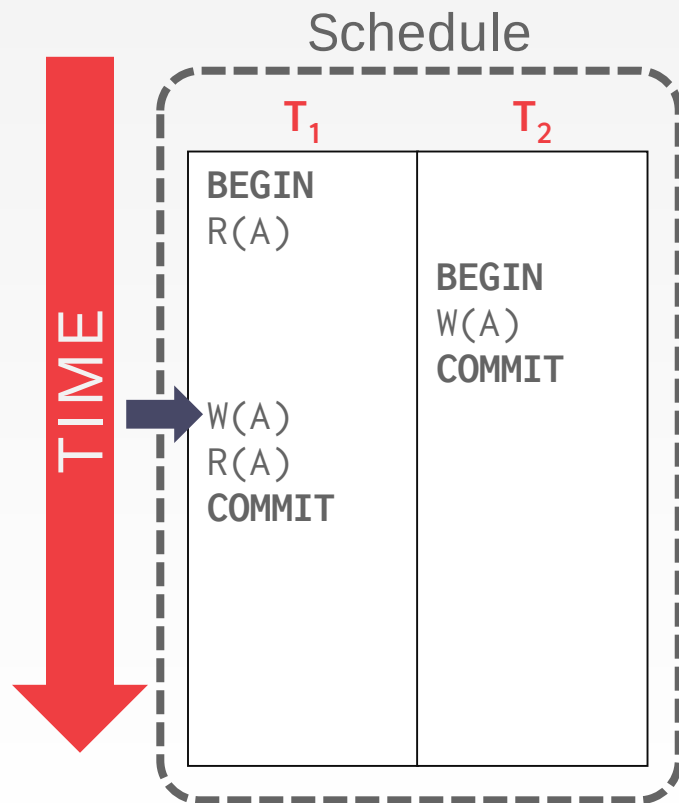
BASIC T/O – EXAMPLE #2



Database

Object	R-TS	W-TS
A	1	2
B	0	0

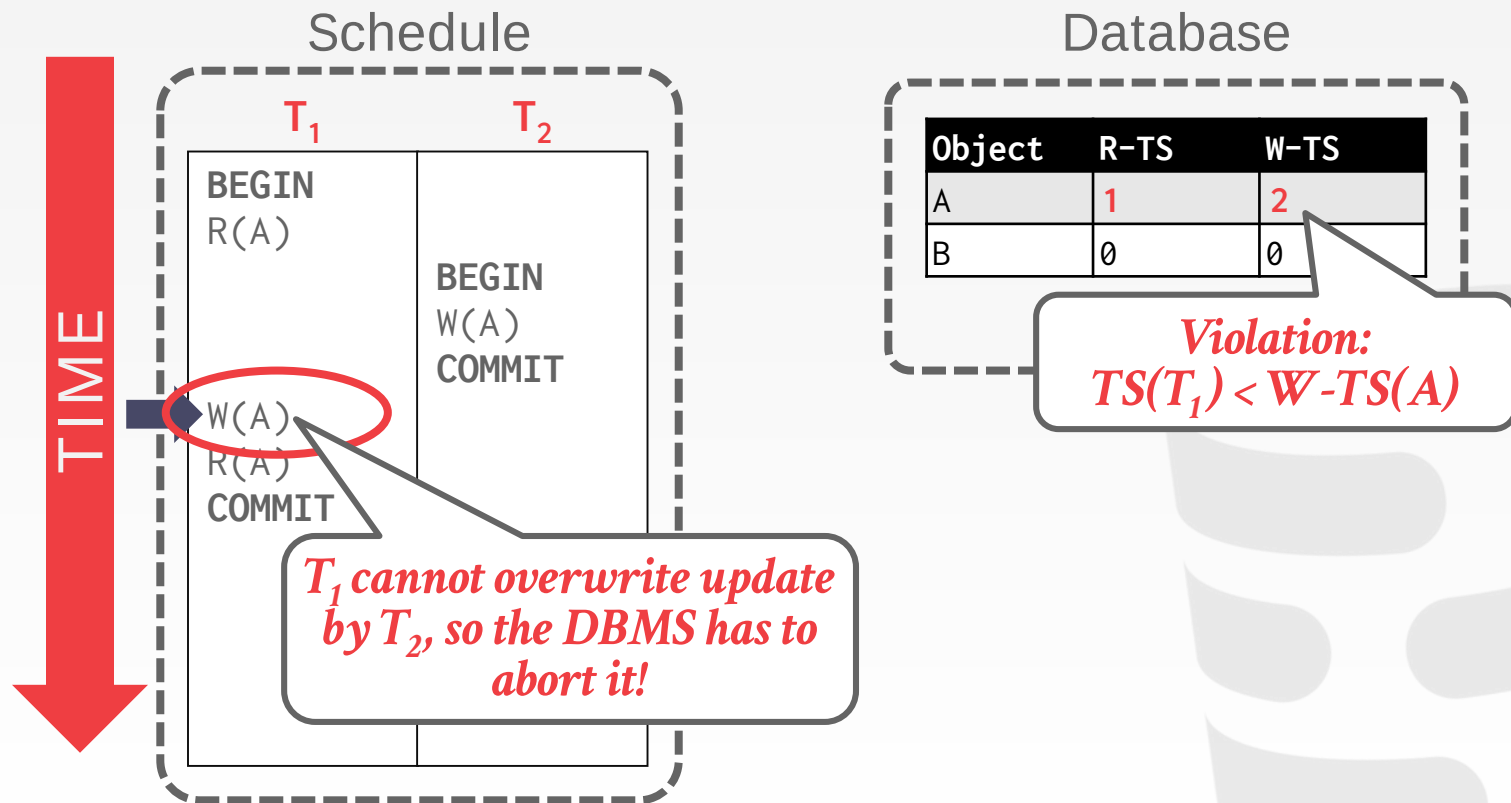
BASIC T/O – EXAMPLE #2



Database

Object	R-TS	W-TS
A	1	2
B	0	0

BASIC T/O – EXAMPLE #2



THOMAS WRITE RULE

If $TS(T_i) < R-TS(X)$:

→ Abort and restart T_i .

If $TS(T_i) < W-TS(X)$:

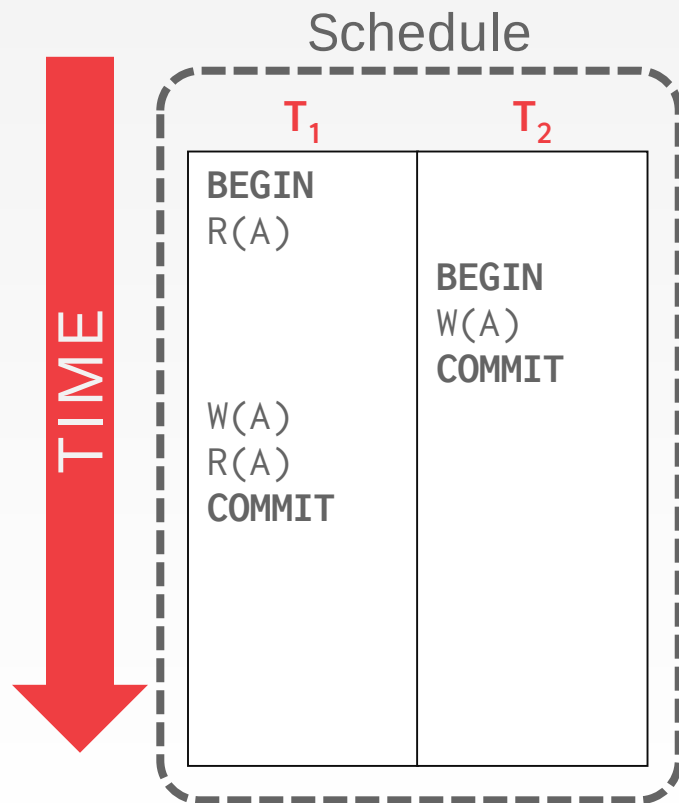
→ Thomas Write Rule: Ignore the write and allow the txn to continue.

→ This violates timestamp order of T_i .

Else:

→ Allow T_i to write X and update $W-TS(X)$

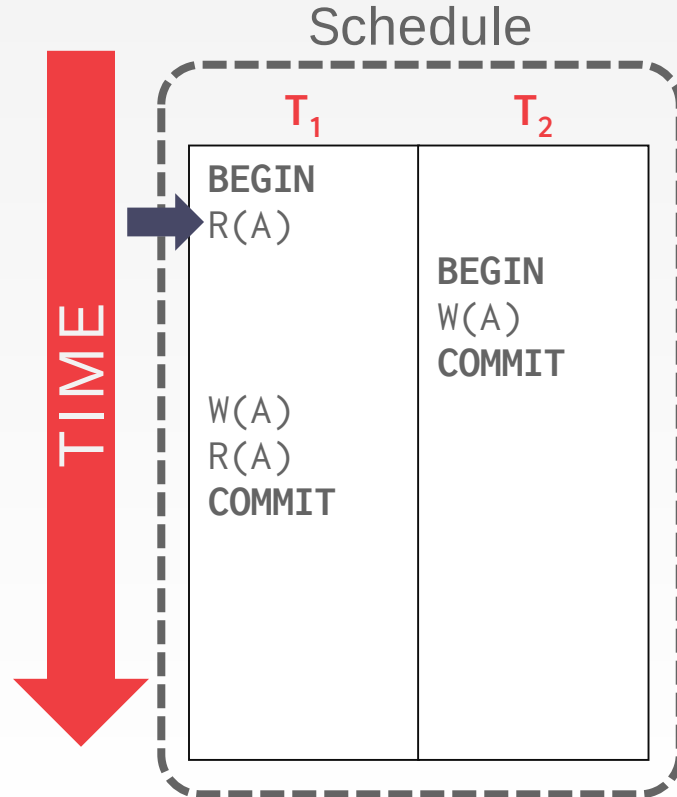
BASIC T/O – EXAMPLE #2



Database

Object	R-TS	W-TS
A	0	0
B	0	0

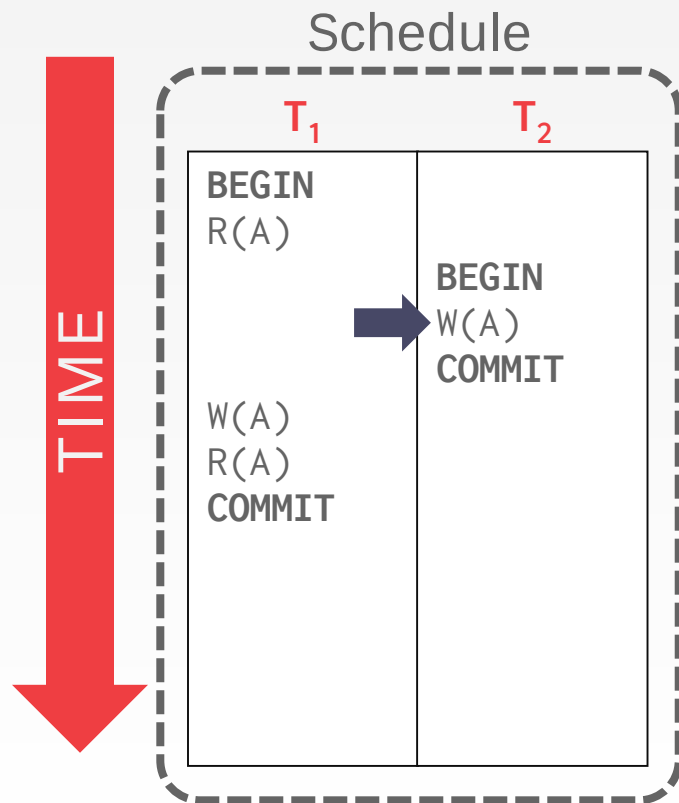
BASIC T/O – EXAMPLE #2



Database

Object	R-TS	W-TS
A	1	0
B	0	0

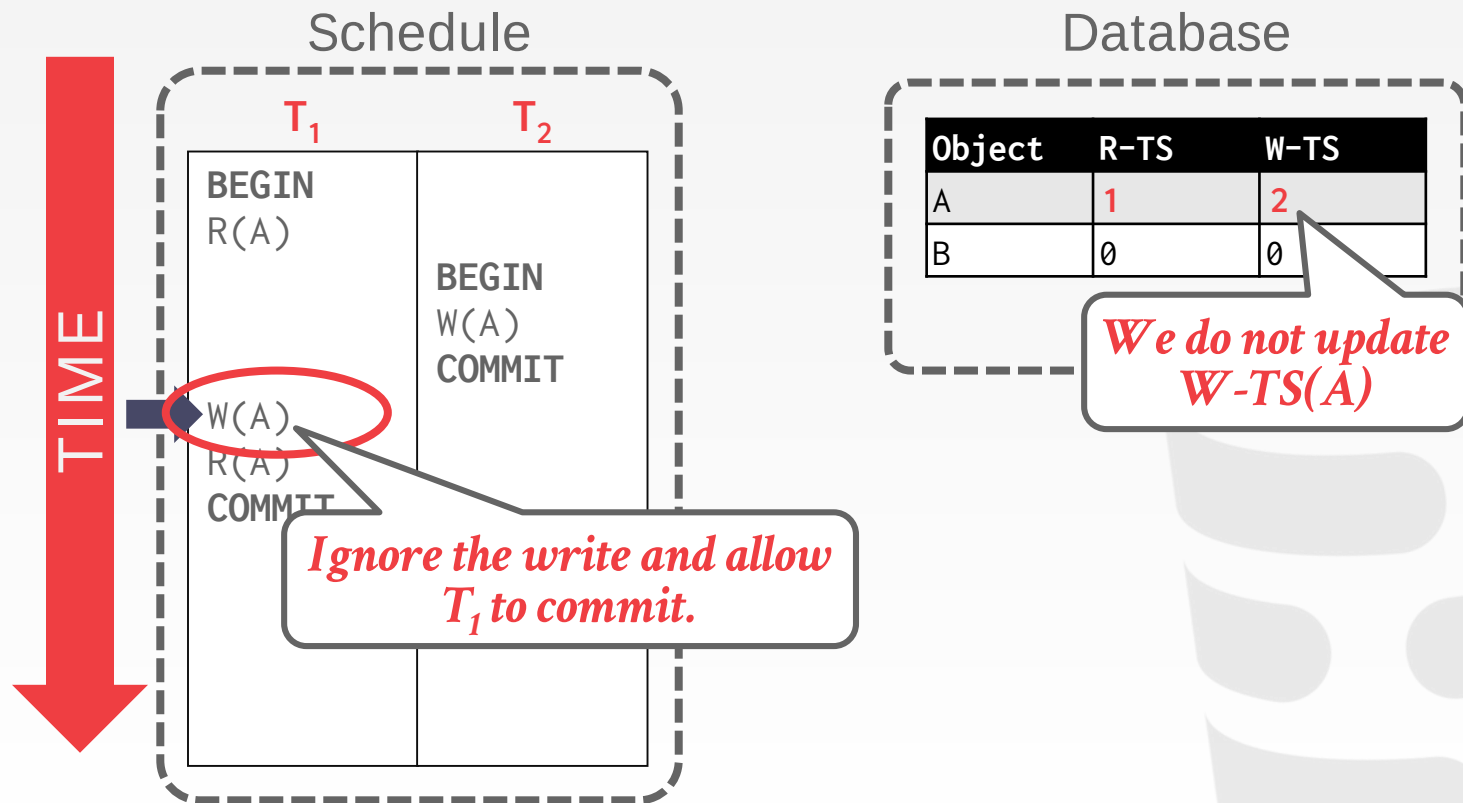
BASIC T/O – EXAMPLE #2



Database

Object	R-TS	W-TS
A	1	2
B	0	0

BASIC T/O – EXAMPLE #2



BASIC T/O

Generates a schedule that is conflict serializable if you do **not** use the Thomas Write Rule.

- No deadlocks because no txn ever waits.
- Possibility of starvation for long txns if short txns keep causing conflicts.

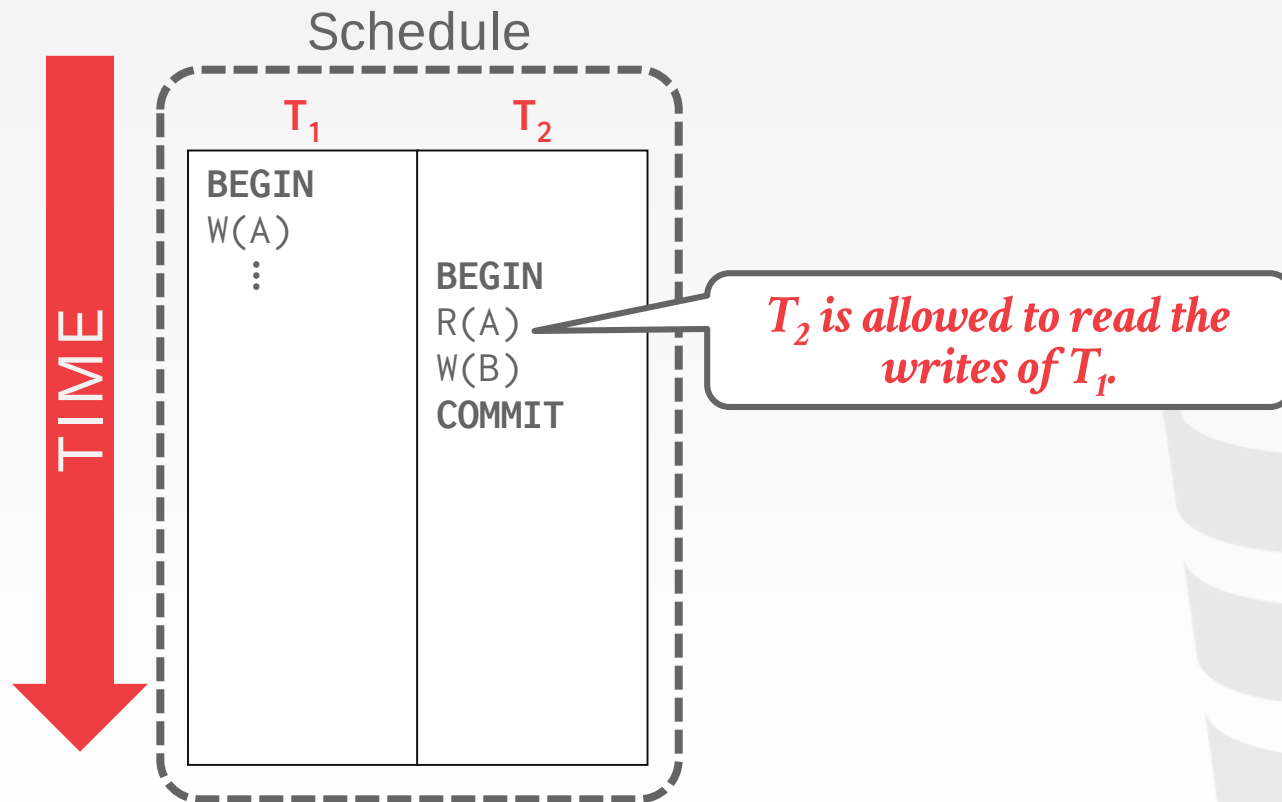
Permits schedules that are not **recoverable**.

RECOVERABLE SCHEDULES

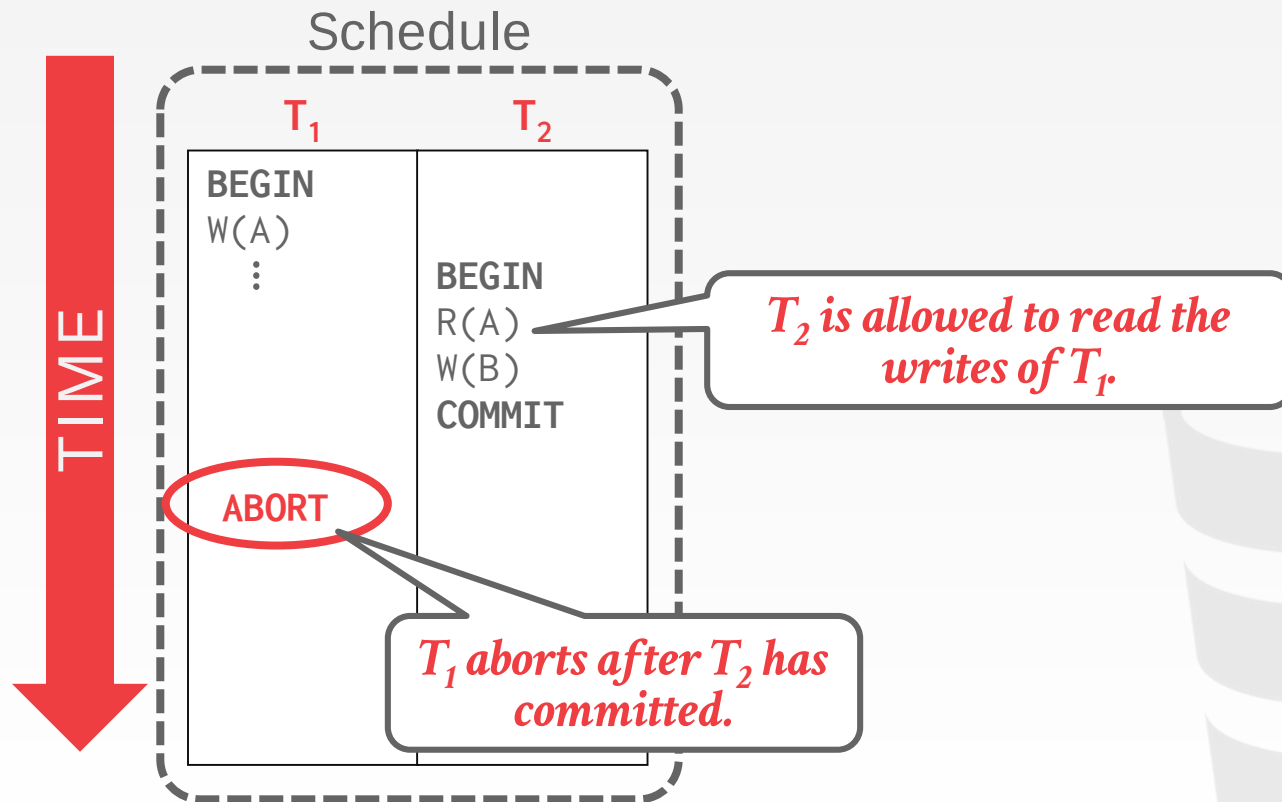
A schedule is **recoverable** if txns commit only after all txns whose changes they read, commit.

Otherwise, the DBMS cannot guarantee that txns read data that will be restored after recovering from a crash.

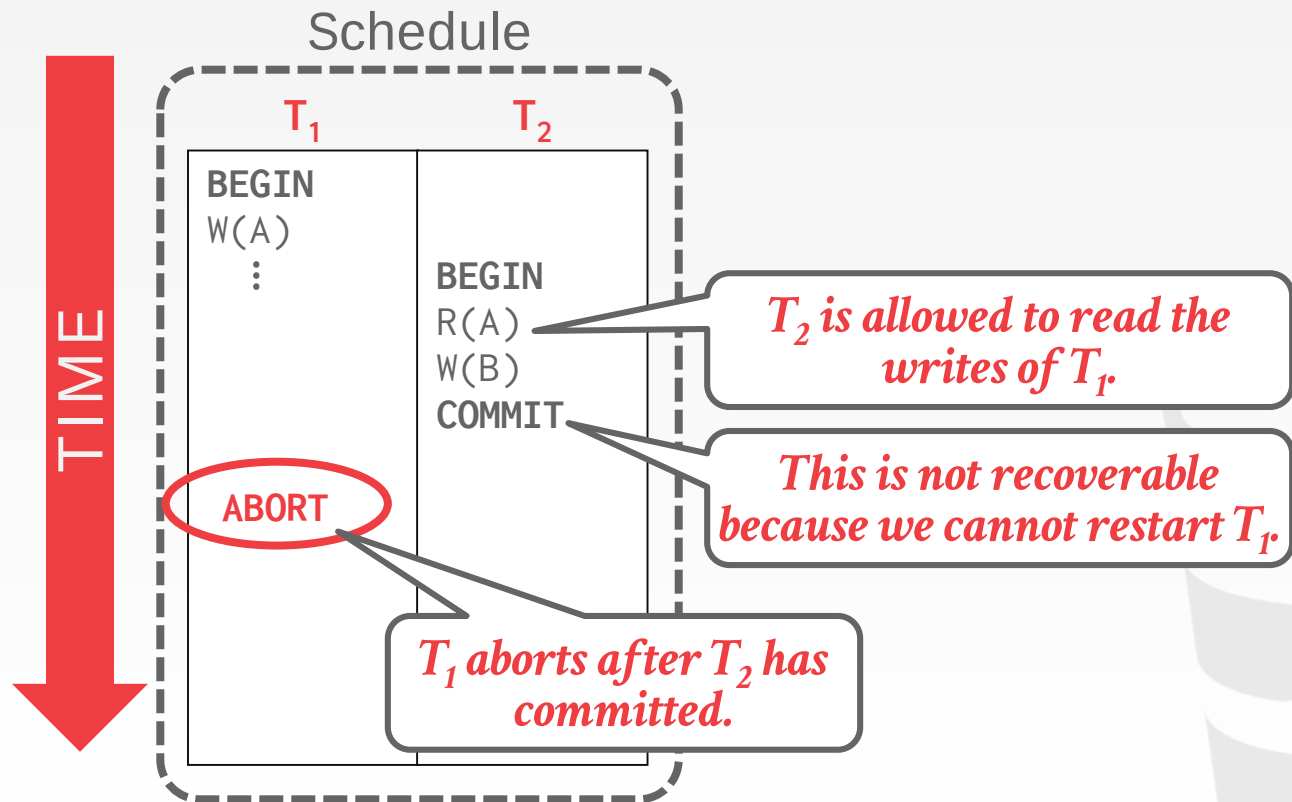
RECOVERABLE SCHEDULES



RECOVERABLE SCHEDULES



RECOVERABLE SCHEDULES



BASIC T/O – PERFORMANCE ISSUES

High overhead from copying data to txn's workspace and from updating timestamps.

Long running txns can get starved.

→ The likelihood that a txn will read something from a newer txn increases.

OBSERVATION

If you assume that conflicts between txns are rare and that most txns are short-lived, then forcing txns to wait to acquire locks adds a lot of overhead.

A better approach is to optimize for the no-conflict case.

OPTIMISTIC CONCURRENCY CONTROL

The DBMS creates a private workspace for each txn.

- Any object read is copied into workspace.
- Modifications are applied to workspace.

When a txn commits, the DBMS compares workspace write set to see whether it conflicts with other txns.

If there are no conflicts, the write set is installed into the "global" database.

On Optimistic Methods for Concurrency Control

H. T. KUNG and JOHN T. ROBINSON
Carnegie-Mellon University

Most current approaches to concurrency control in database systems rely on locking of data objects as a control mechanism. In this paper, two families of nonlocking concurrency controls are presented. The methods used are "optimistic" in the sense that they rely mainly on transaction backup as a control mechanism, "hoping" that conflicts between transactions will not occur. Applications for which these methods should be more efficient than locking are discussed.

Key Words and Phrases: databases, concurrency controls, transaction processing
CR Categories: 4.32, 4.33

1. INTRODUCTION

Consider the problem of providing shared access to a database organized as a collection of objects. We assume that certain distinguished objects, called the roots, are always present and access to any object other than a root is gained only by first accessing a root and then following pointers to that object. Any sequence of accesses to the database that preserves the integrity constraints of the data is called a *transaction* (see, e.g., [4]).

If our goal is to maximize the throughput of accesses to the database, then there are at least two cases where highly concurrent access is desirable.

- (1) The amount of data is sufficiently great that at any given time only a fraction of the database can be present in primary memory, so that it is necessary to swap parts of the database from secondary memory as needed.
- (2) Even if the entire database can be present in primary memory, there may be multiple processors.

In both cases the hardware will be underutilized if the degree of concurrency is too low.

However, as is well known, unrestricted concurrent access to a shared database will, in general, cause the integrity of the database to be lost. Most current

Permission to copy without fee all or part of this material is granted provided that the copies are not made or distributed for direct commercial advantage, the ACM copyright notice and the title of the publication and its date appear, and notice is given that copying is by permission of the Association for Computing Machinery. To copy otherwise, or to republish, requires a fee and/or specific permission.

This research was supported in part by the National Science Foundation under Grant MCS 78-23676 and the Office of Naval Research under Contract N00014-76-C-0370.

Authors' address: Department of Computer Science, Carnegie-Mellon University, Pittsburgh, PA 15213.

© 1981 ACM 0362-5915/81/0000-0213 \$00.75

ACM Transactions on Database Systems, Vol. 6, No. 2, June 1981, Pages 213-226.

OCC PHASES

#1 – Read Phase:

- Track the read/write sets of txns and store their writes in a private workspace.

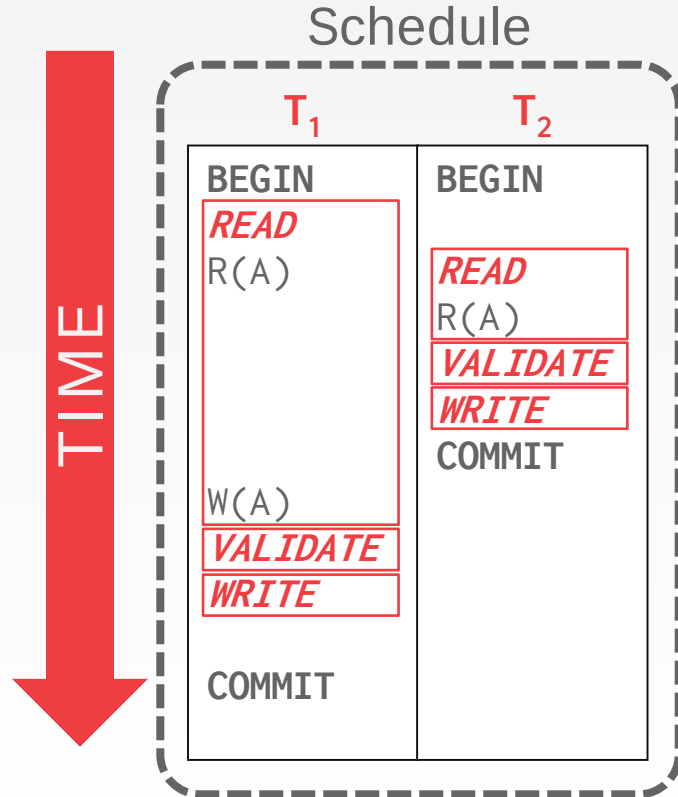
#2 – Validation Phase:

- When a txn commits, check whether it conflicts with other txns.

#3 – Write Phase:

- If validation succeeds, apply private changes to database. Otherwise abort and restart the txn.

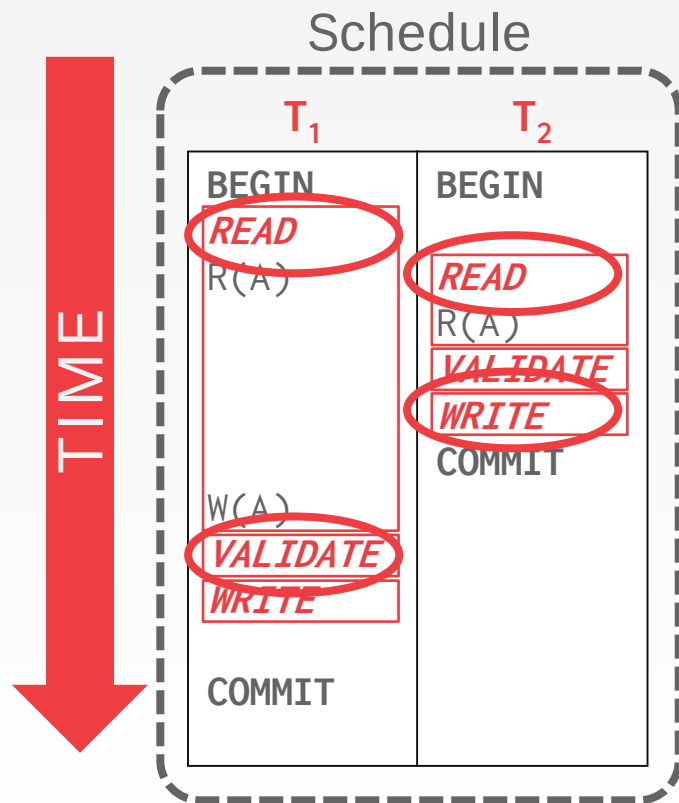
OCC – EXAMPLE



Database

Object	Value	W-TS
A	123	0
-	-	-

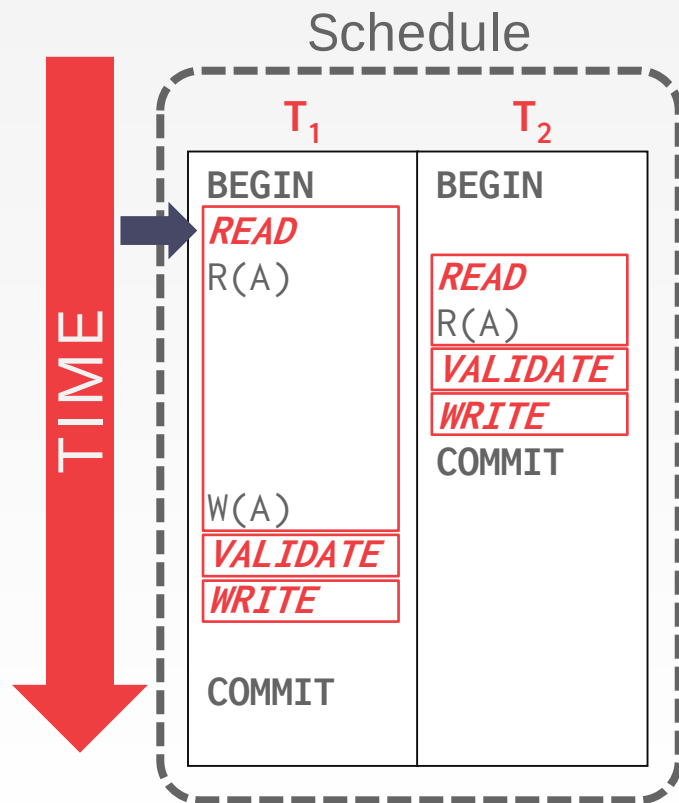
OCC – EXAMPLE



Database

Object	Value	W-TS
A	123	0
-	-	-

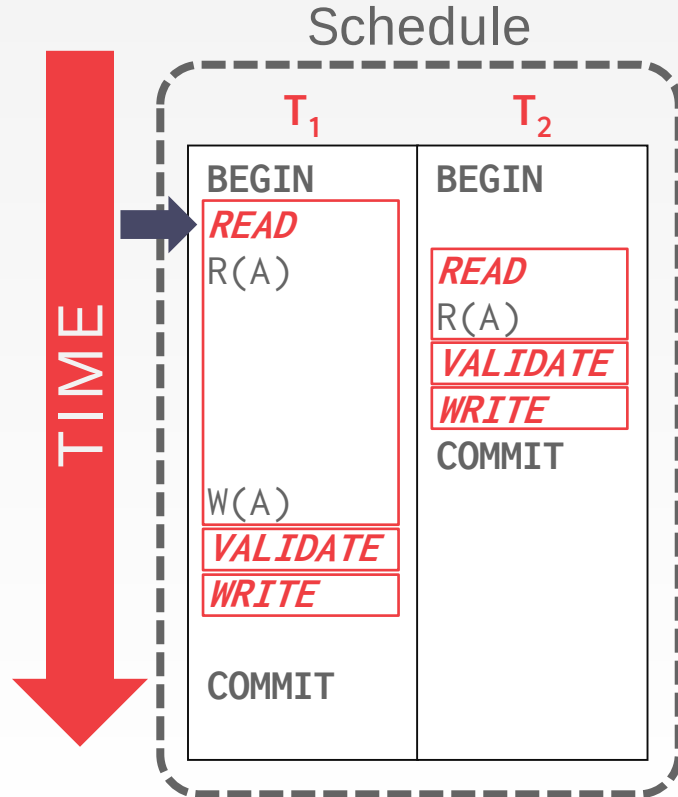
OCC – EXAMPLE



Database

Object	Value	W-TS
A	123	0
-	-	-

OCC – EXAMPLE



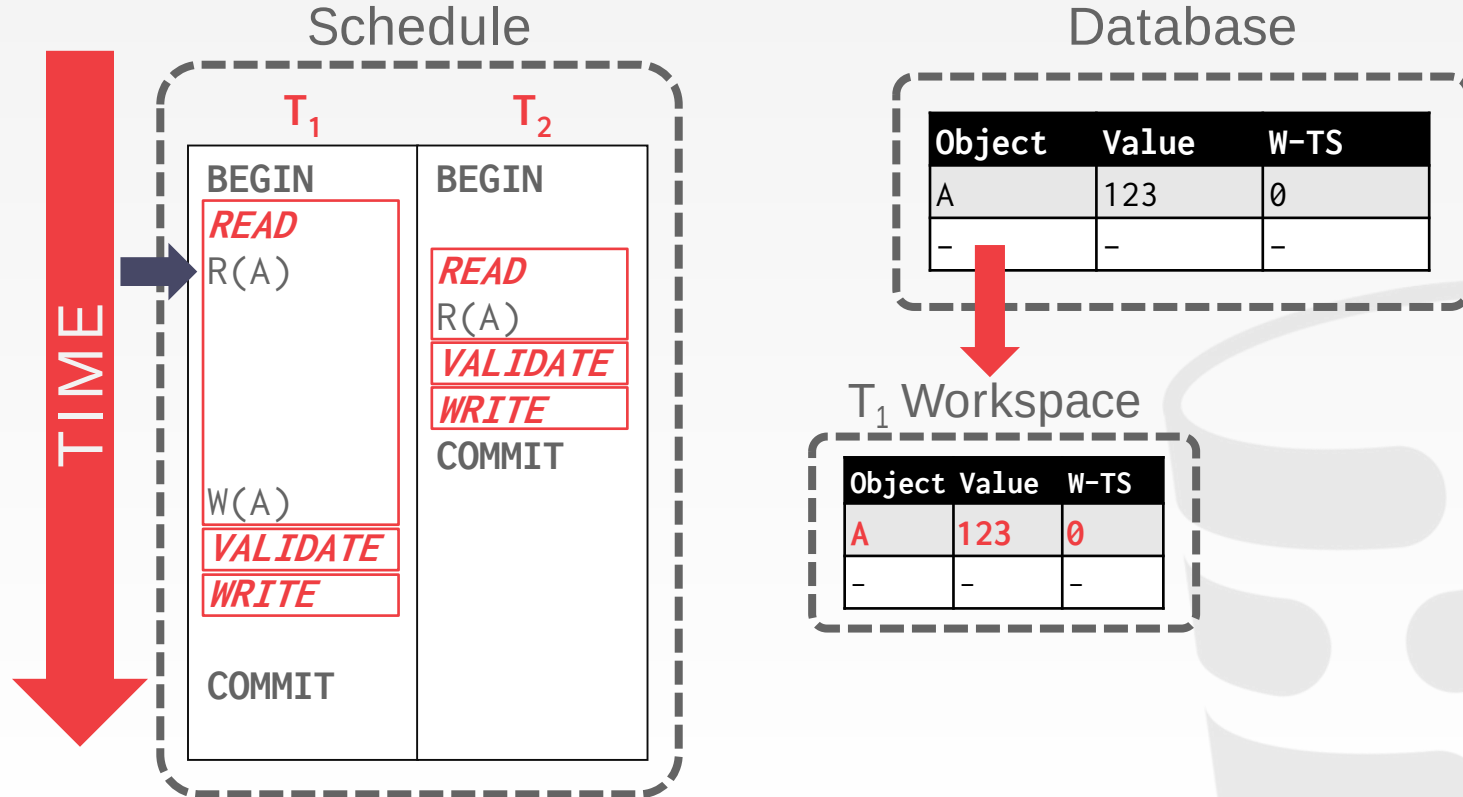
Database

Object	Value	W-TS
A	123	0
-	-	-

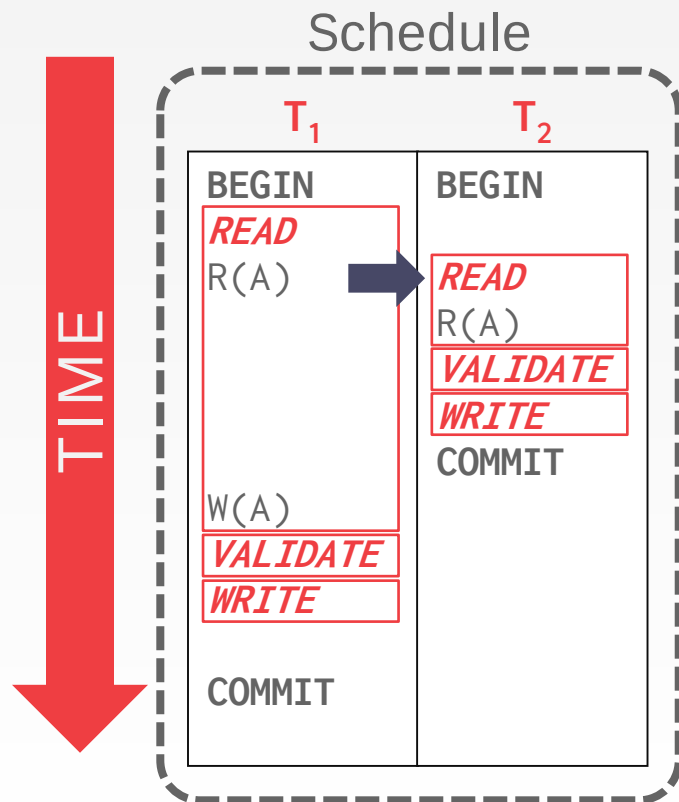
T_1 Workspace

Object	Value	W-TS
-	-	-
-	-	-

OCC – EXAMPLE



OCC – EXAMPLE



Database

Object	Value	W-TS
A	123	0
-	-	-

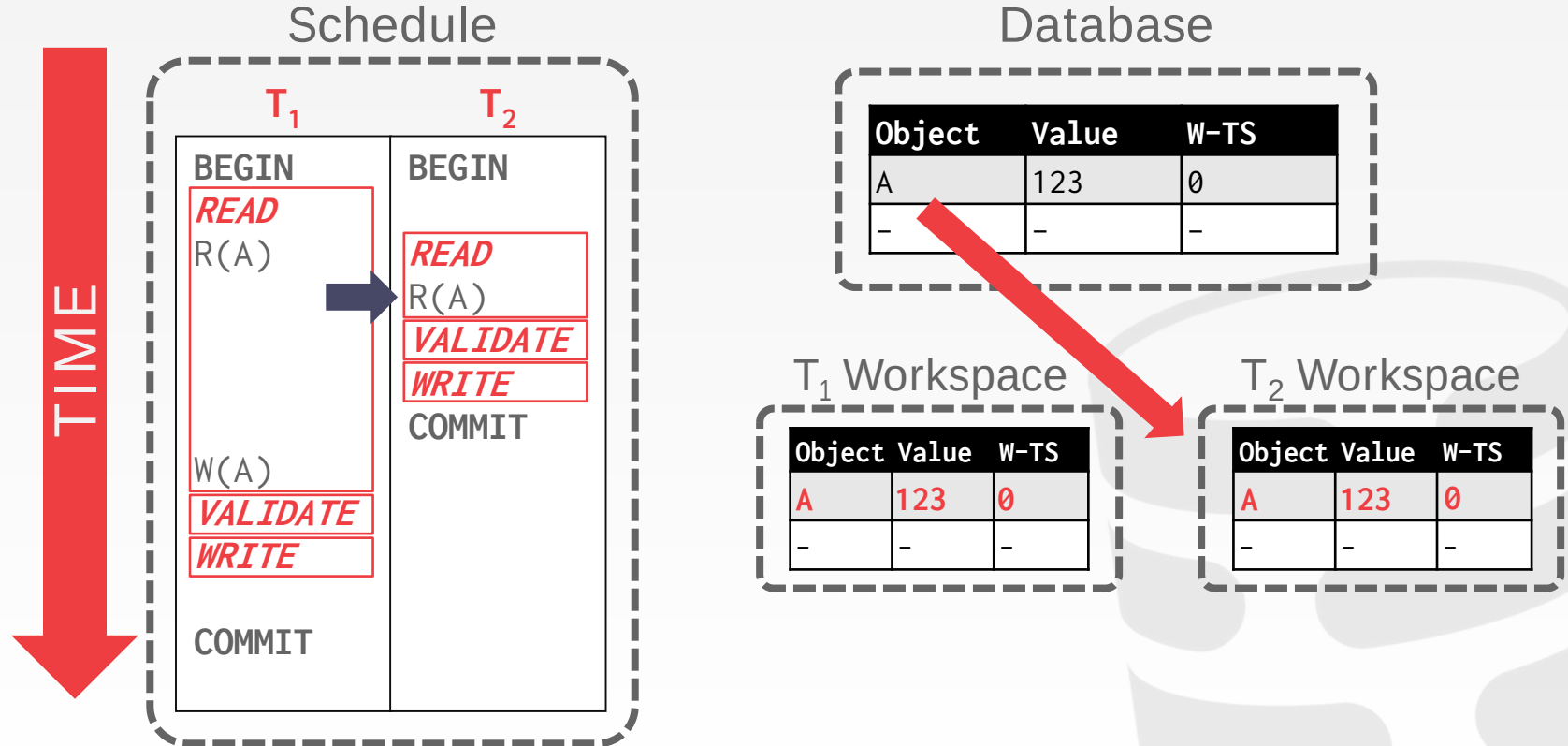
T₁ Workspace

Object	Value	W-TS
A	123	0
-	-	-

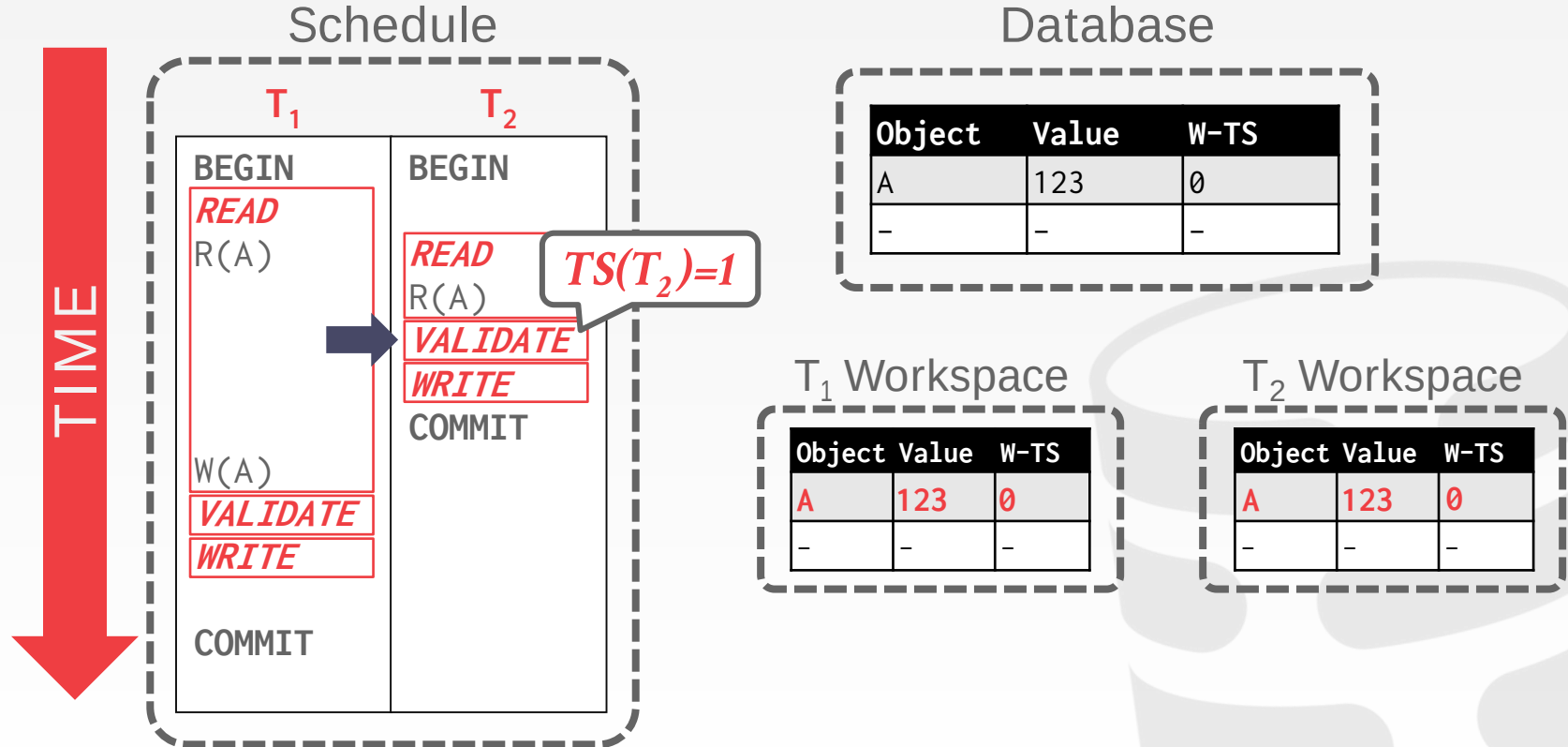
T₂ Workspace

Object	Value	W-TS
-	-	-
-	-	-

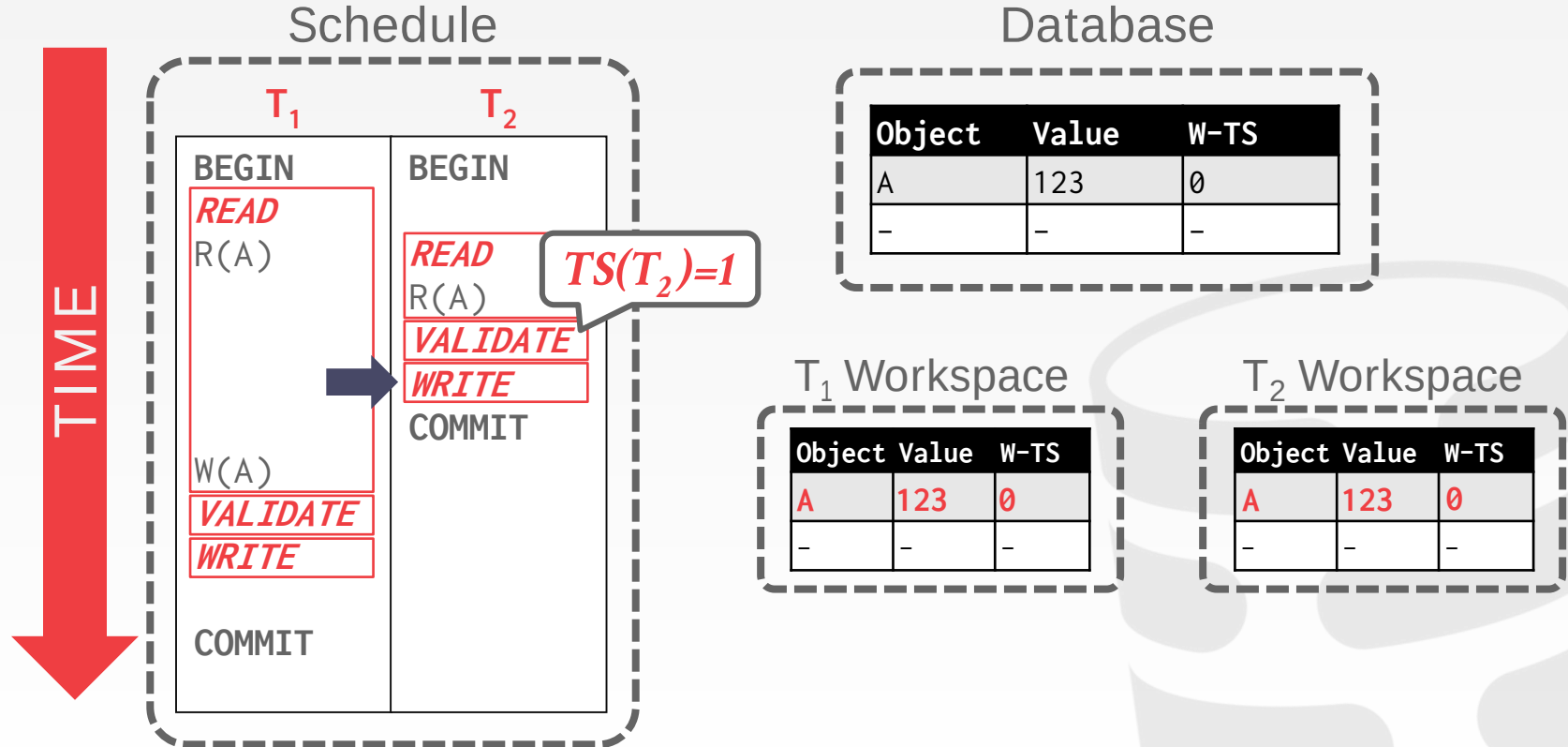
OCC – EXAMPLE



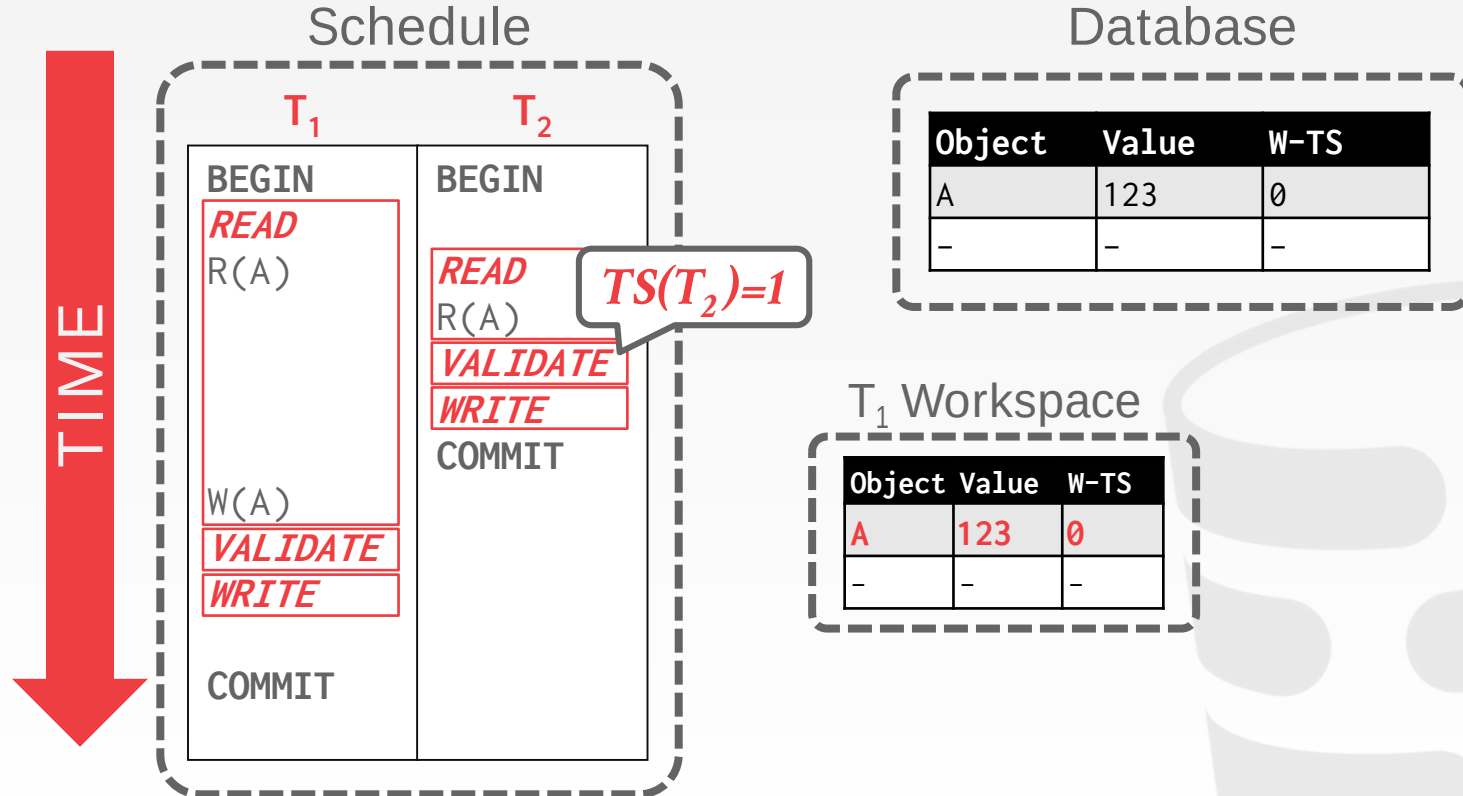
OCC – EXAMPLE



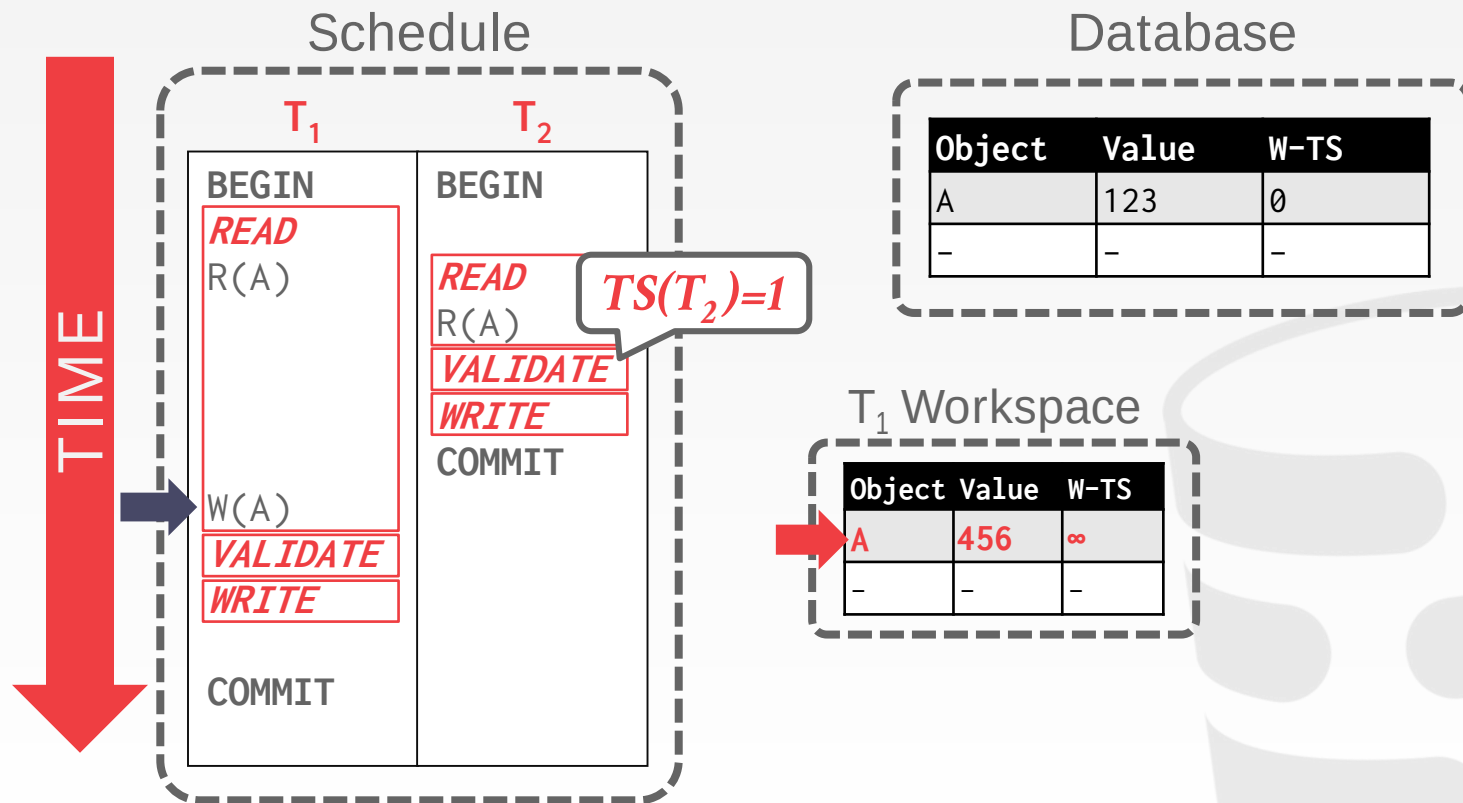
OCC – EXAMPLE



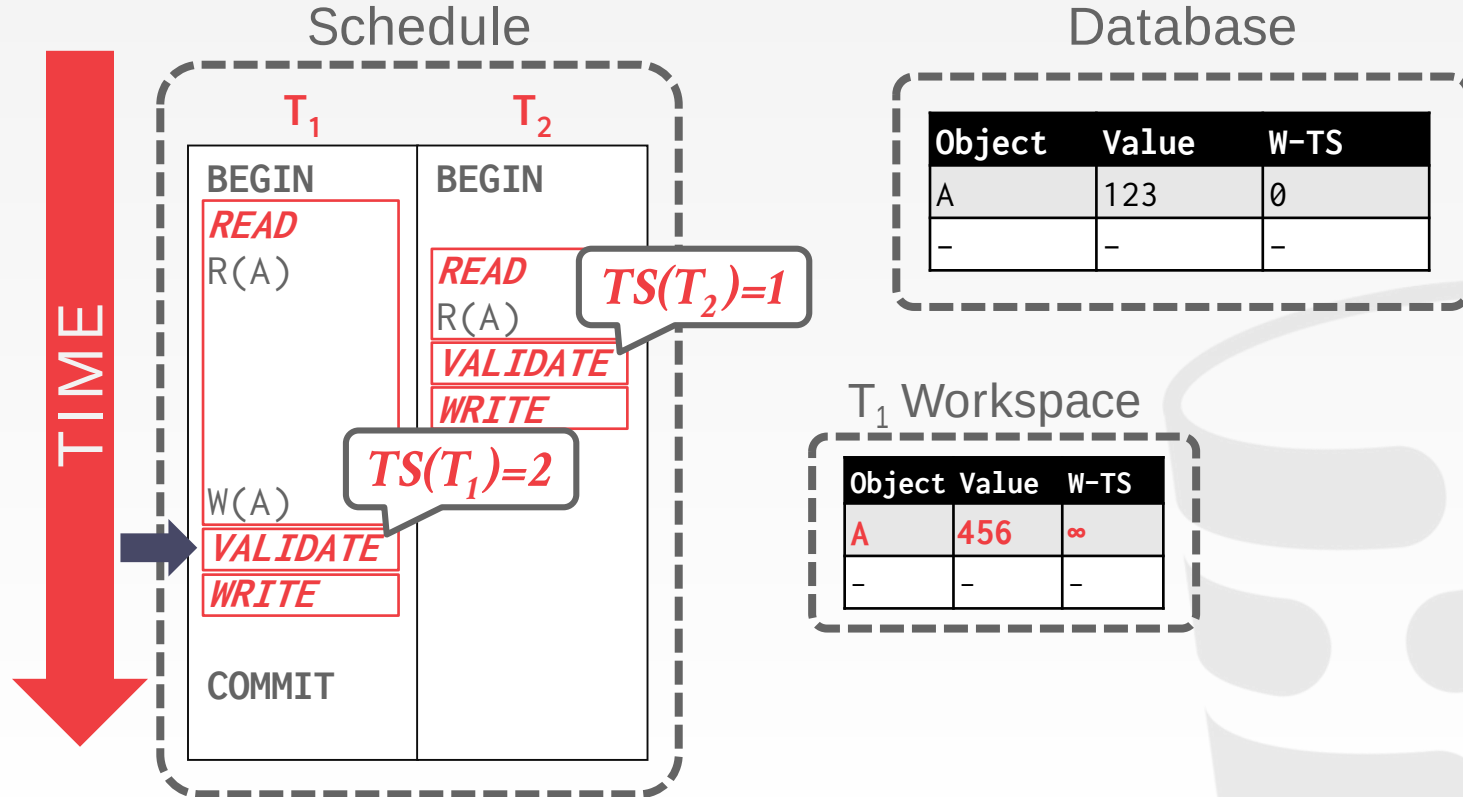
OCC – EXAMPLE



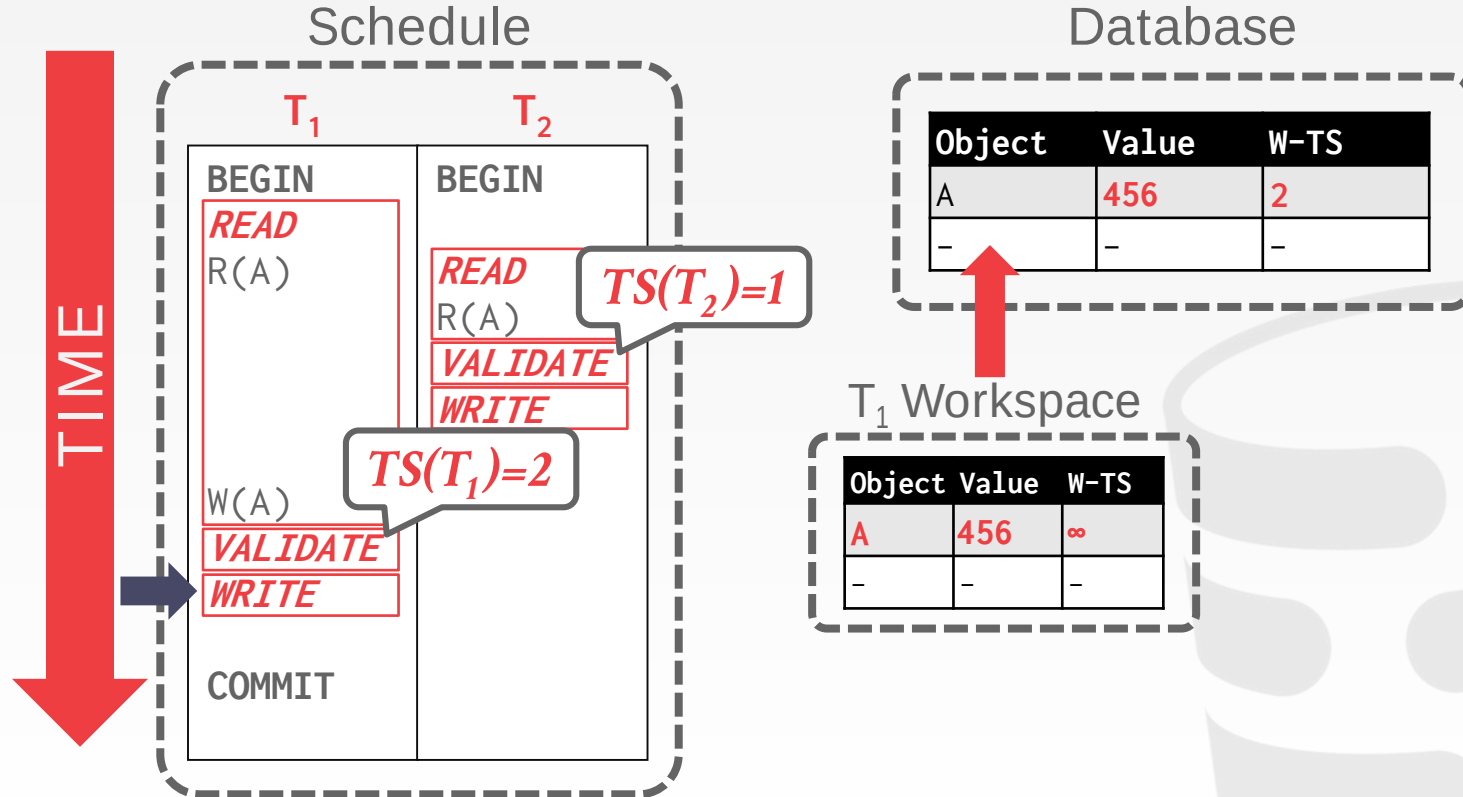
OCC – EXAMPLE



OCC – EXAMPLE



OCC – EXAMPLE



OCC – VALIDATION PHASE

The DBMS needs to guarantee only serializable schedules are permitted.

T_i checks other txns for RW and WW conflicts and makes sure that all conflicts go one way (from older txns to younger txns).

OCC – SERIAL VALIDATION

Maintain global view of all active txns.

Record read set and write set while txns are running and write into private workspace.

Execute **Validation** and **Write** phase inside a protected critical section.

OCC – READ PHASE

Track the read/write sets of txns and store their writes in a private workspace.

The DBMS copies every tuple that the txn accesses from the shared database to its workspace ensure repeatable reads.

OCC – VALIDATION PHASE

Each txn's timestamp is assigned at the beginning of the validation phase.

Check the timestamp ordering of the committing txn with all other running txns.

If $TS(T_i) < TS(T_j)$, then one of the following three conditions must hold...

OCC – VALIDATION PHASE

When the txn invokes **COMMIT**, the DBMS checks if it conflicts with other txns.

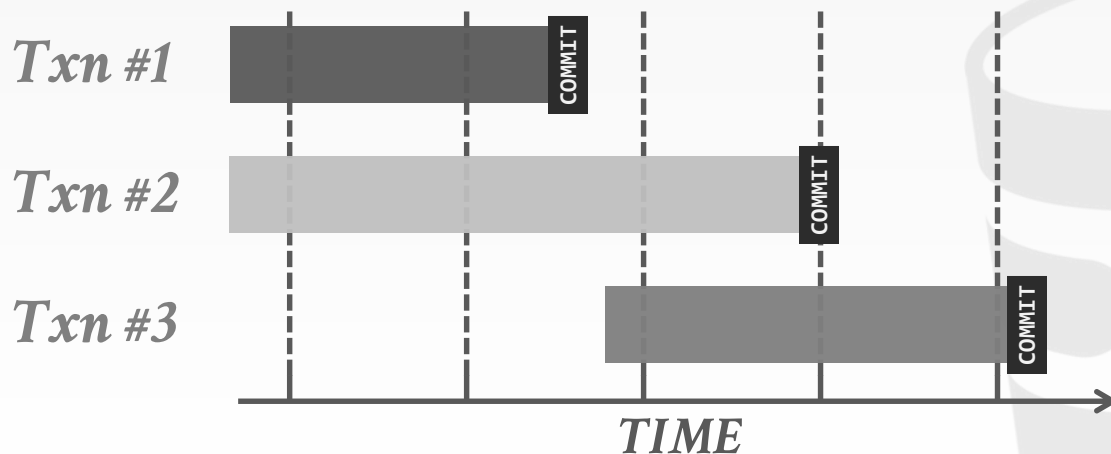
Two methods for this phase:

- Backward Validation
- Forward Validation



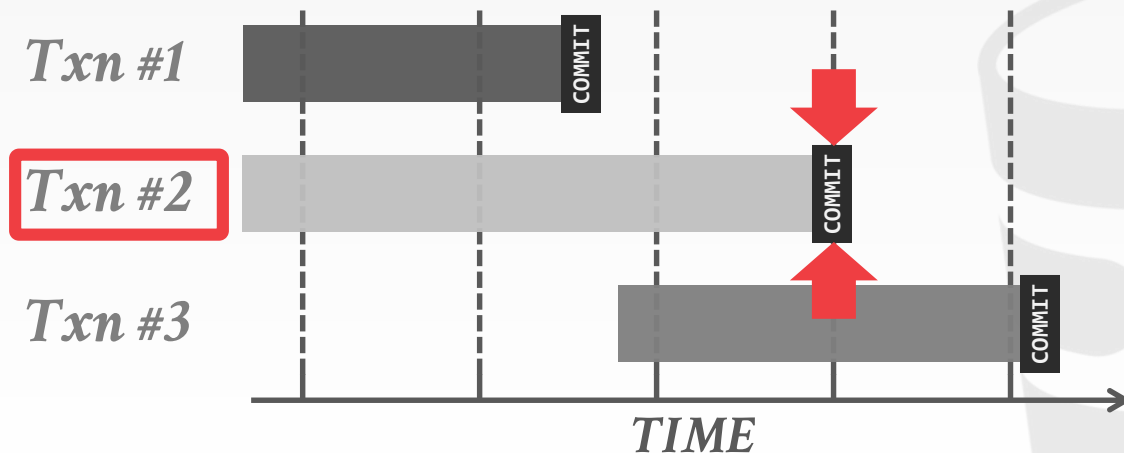
OCC – BACKWARD VALIDATION

Check whether the committing txn intersects its read/write sets with those of any txns that have already committed.



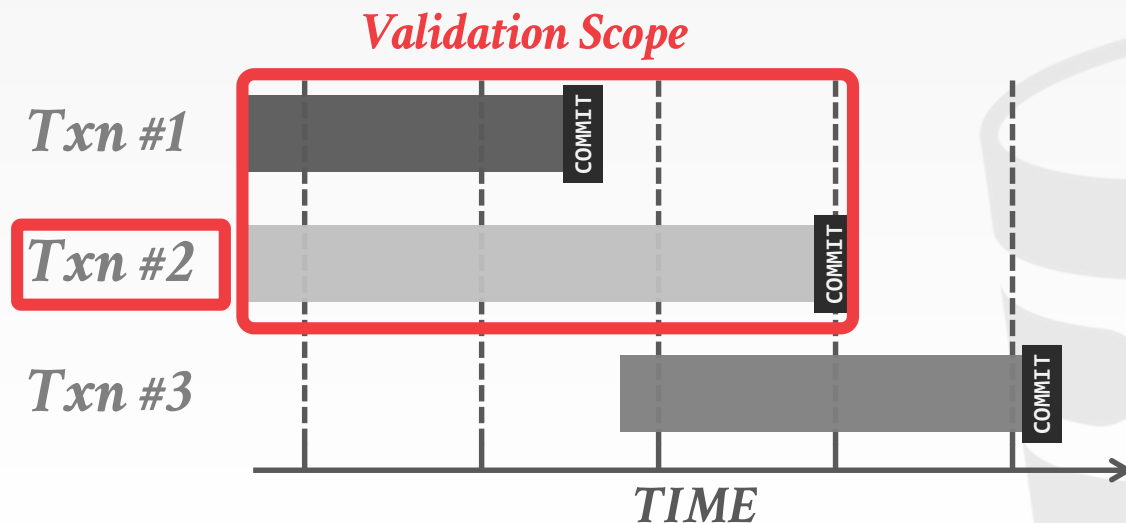
OCC – BACKWARD VALIDATION

Check whether the committing txn intersects its read/write sets with those of any txns that have already committed.



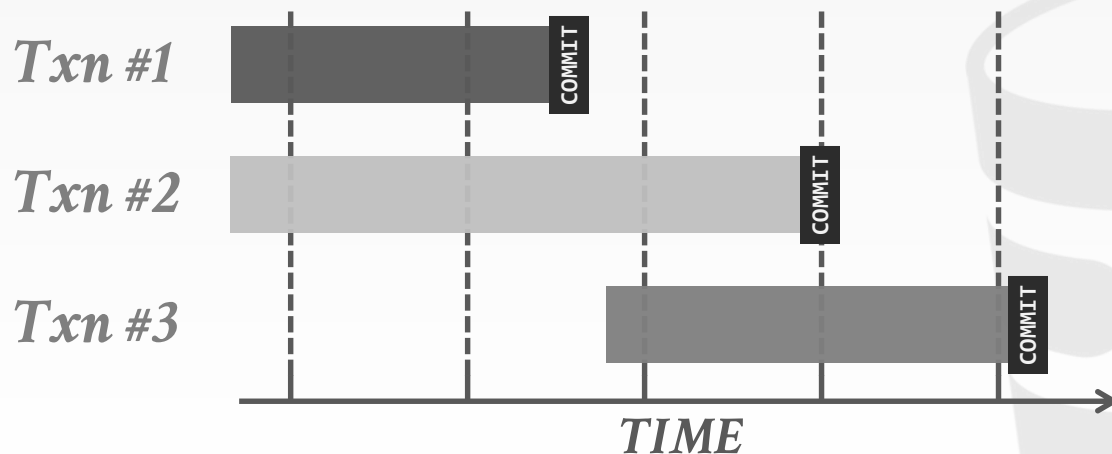
OCC – BACKWARD VALIDATION

Check whether the committing txn intersects its read/write sets with those of any txns that have already committed.



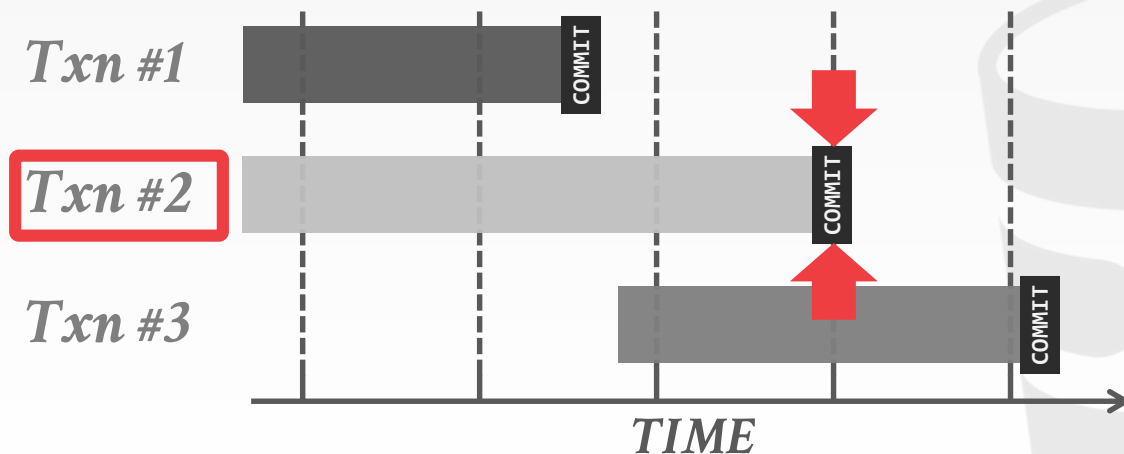
OCC – FORWARD VALIDATION

Check whether the committing txn intersects its read/write sets with any active txns that have not yet committed.



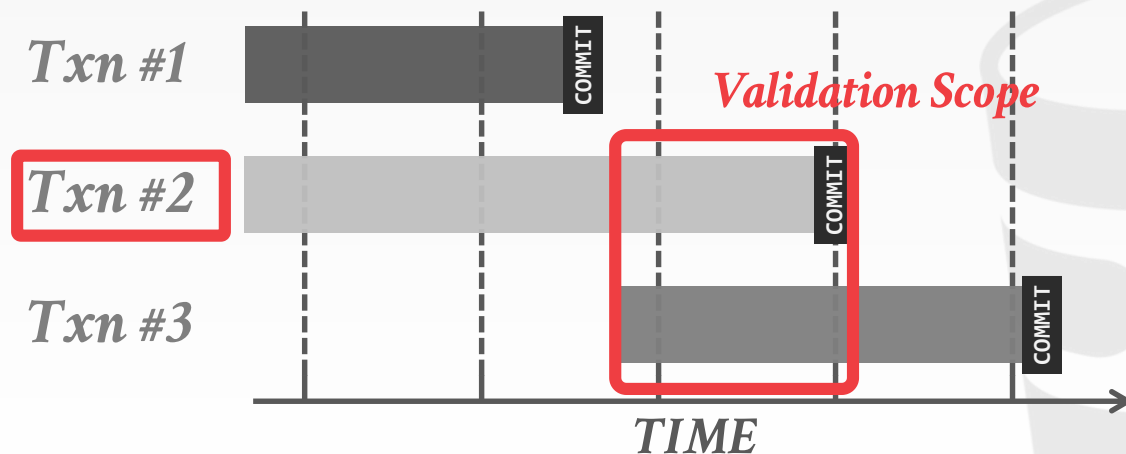
OCC – FORWARD VALIDATION

Check whether the committing txn intersects its read/write sets with any active txns that have not yet committed.



OCC – FORWARD VALIDATION

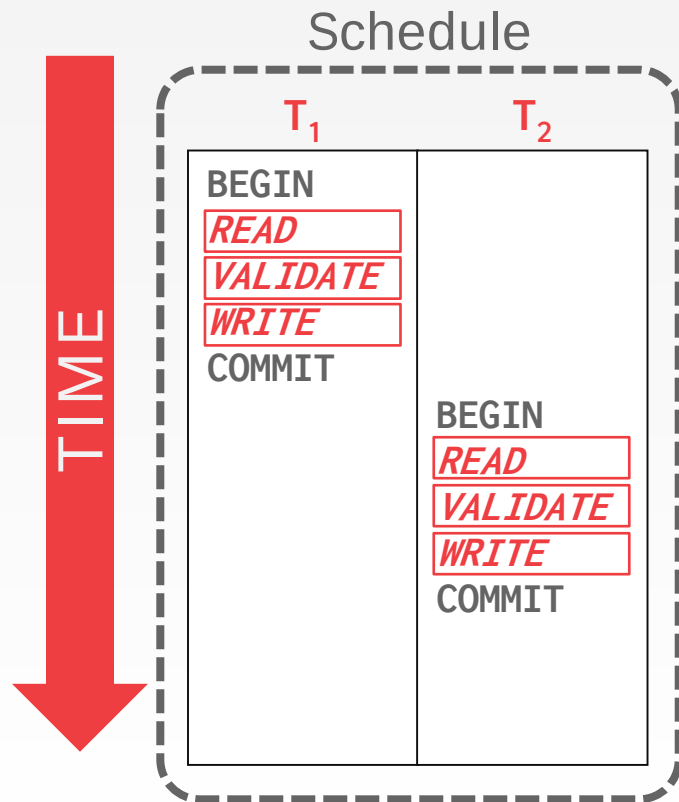
Check whether the committing txn intersects its read/write sets with any active txns that have not yet committed.



OCC – VALIDATION STEP #1

T_i completes all three phases before T_j begins.

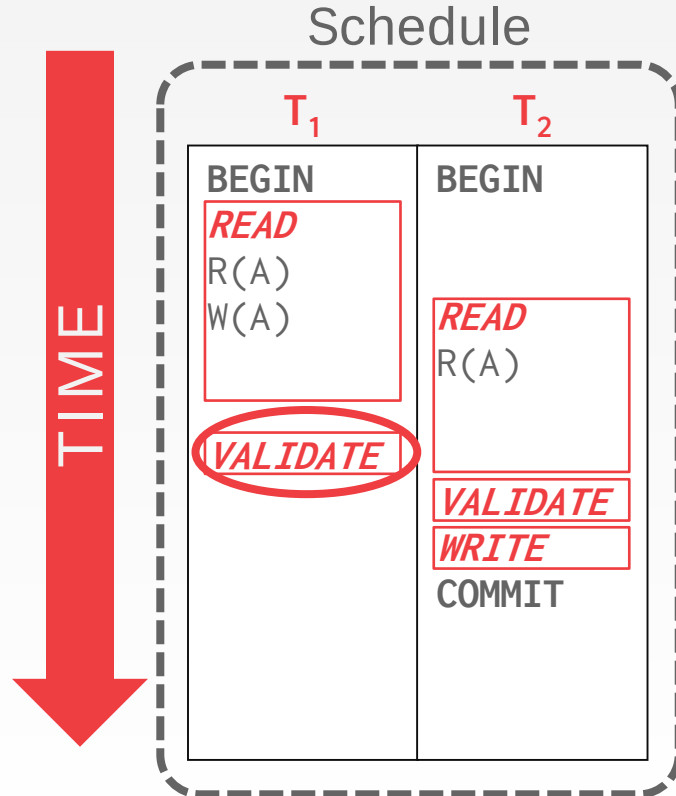
OCC – VALIDATION STEP #1



OCC – VALIDATION STEP #2

T_i completes before T_j starts its **Write** phase, and
 T_i does not write to any object read by T_j .
→ $\text{WriteSet}(T_i) \cap \text{ReadSet}(T_j) = \emptyset$

OCC – VALIDATION STEP #2



Database

Object	Value	W-TS
A	123	0
-	-	-

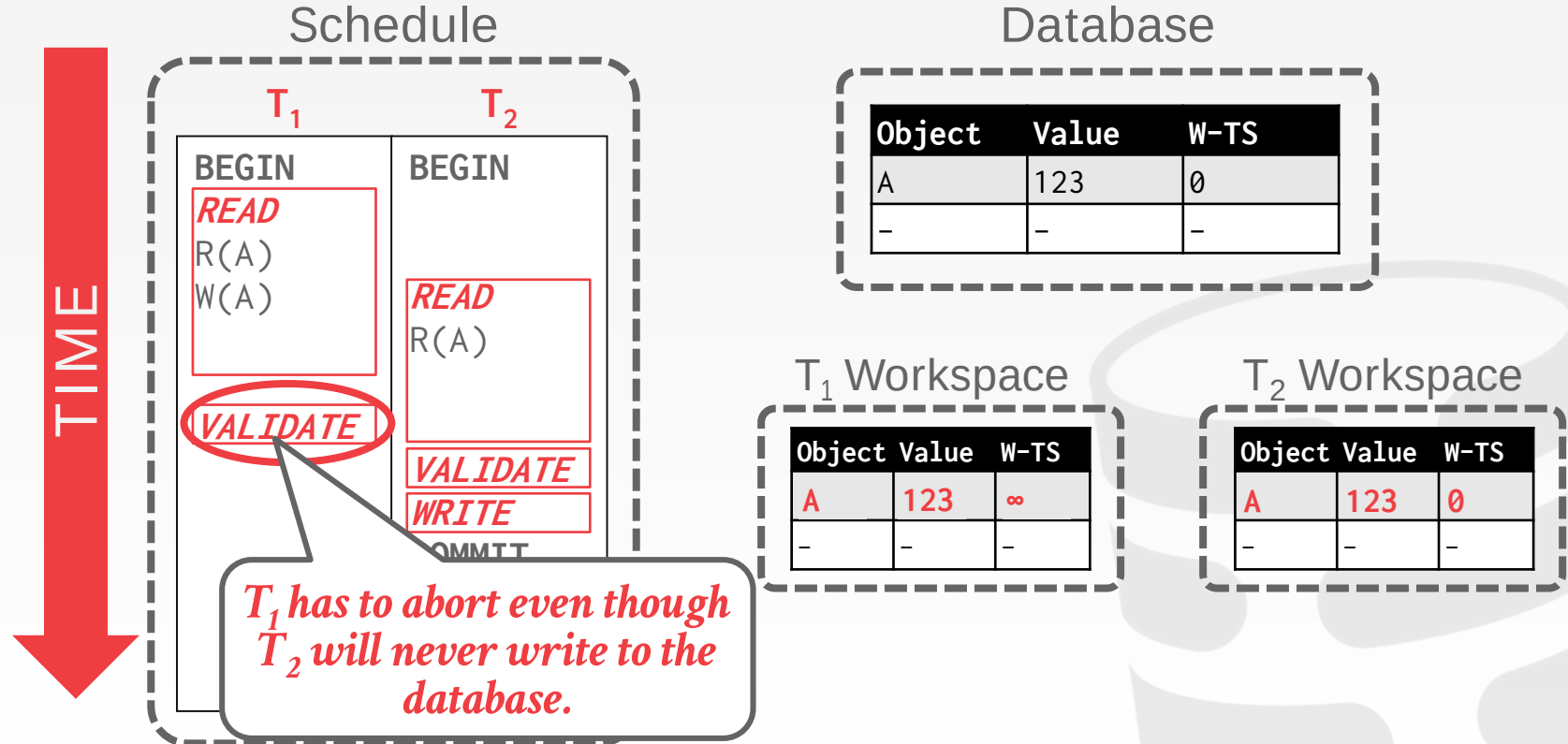
T_1 Workspace

Object	Value	W-TS
A	123	∞
-	-	-

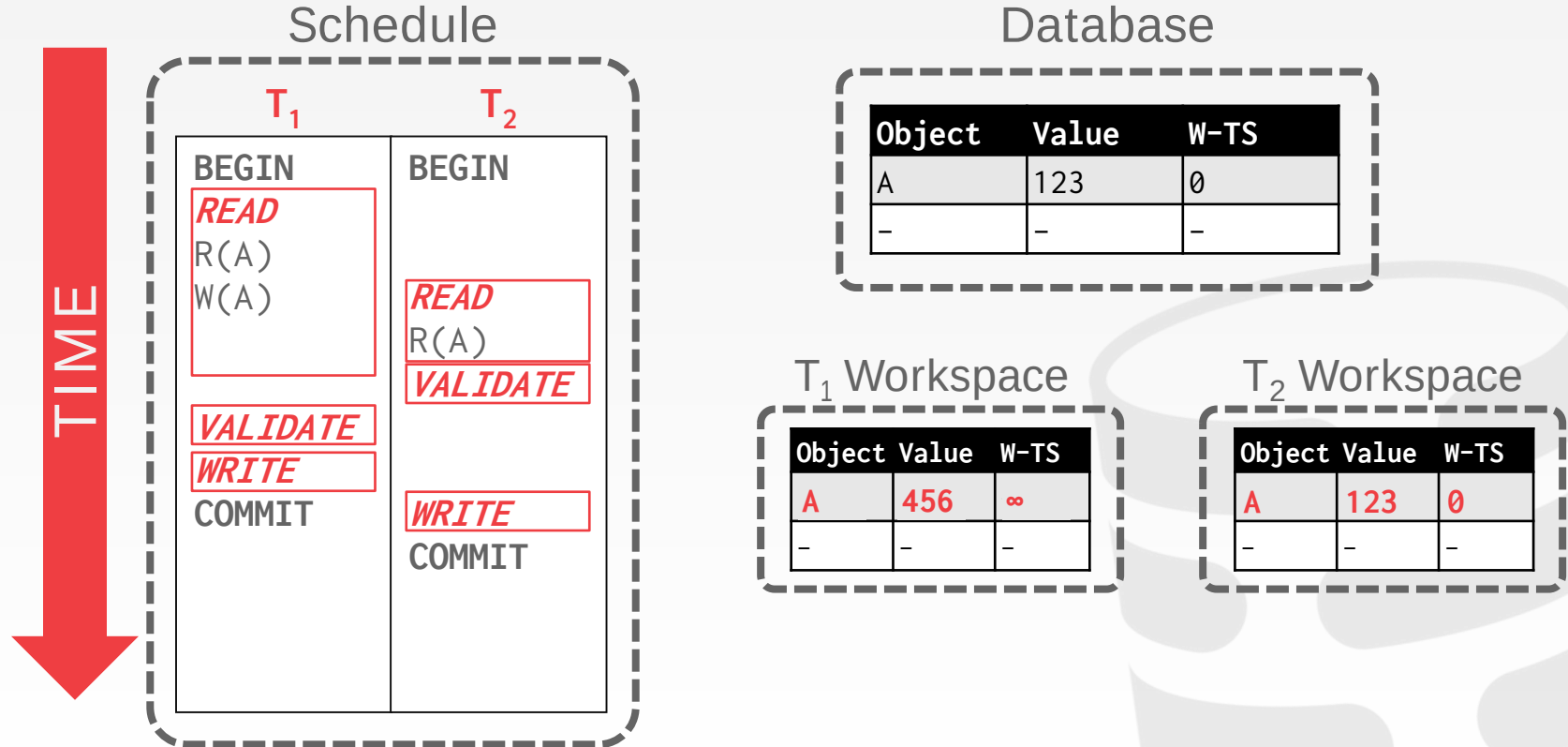
T_2 Workspace

Object	Value	W-TS
A	123	0
-	-	-

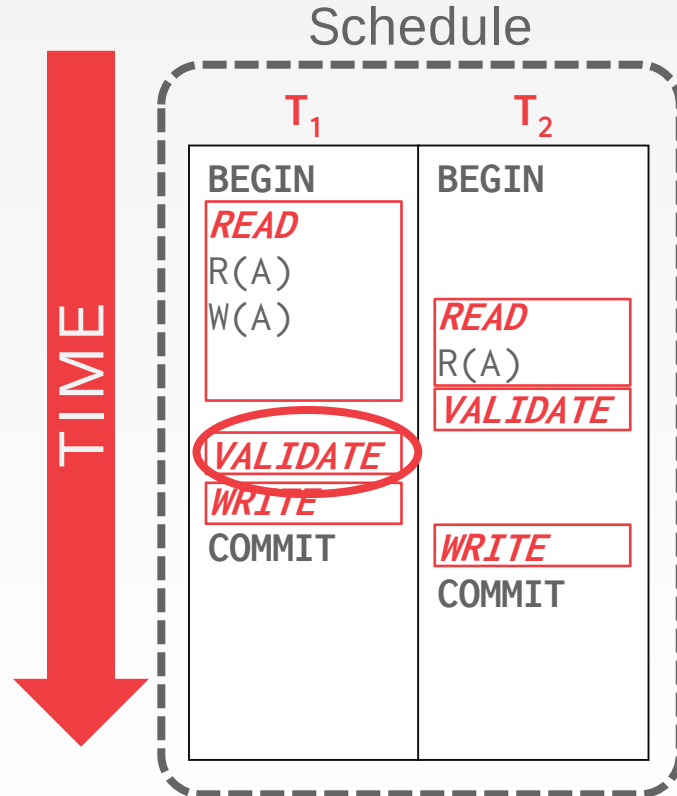
OCC – VALIDATION STEP #2



OCC – VALIDATION STEP #2



OCC – VALIDATION STEP #2



Database

Object	Value	W-TS
A	123	0
-	-	-

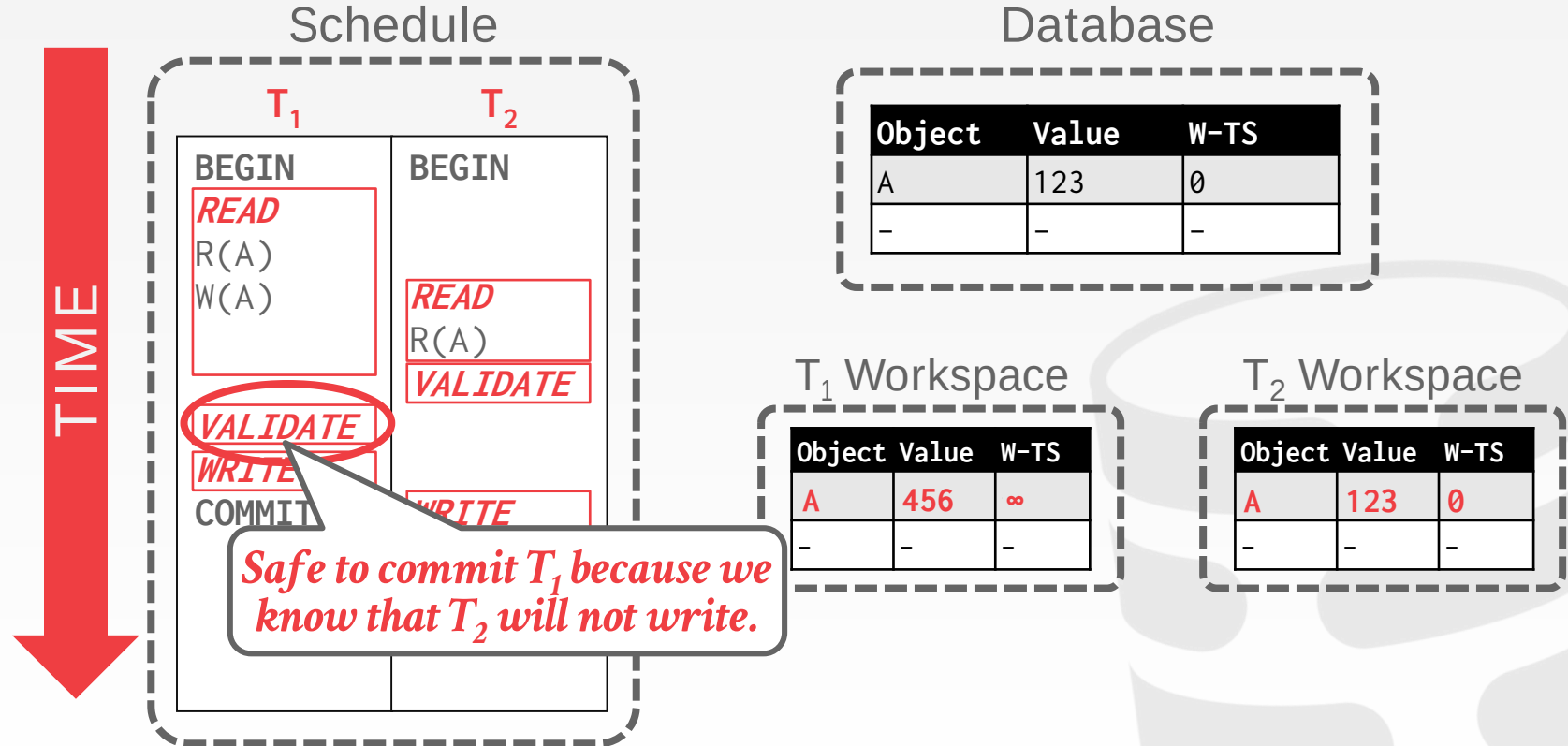
T_1 Workspace

Object	Value	W-TS
A	456	∞
-	-	-

T_2 Workspace

Object	Value	W-TS
A	123	0
-	-	-

OCC – VALIDATION STEP #2



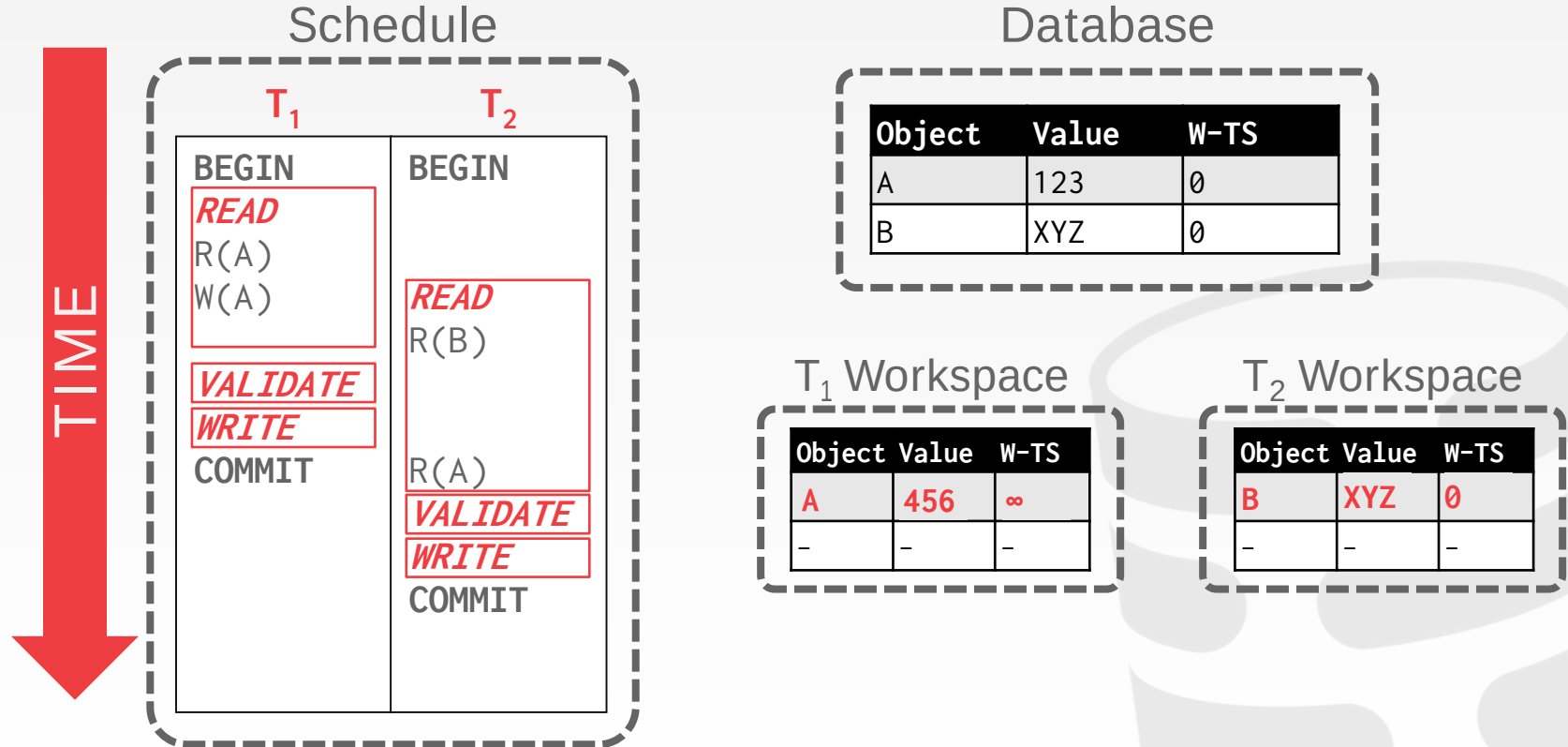
OCC – VALIDATION STEP #3

T_i completes its **Read** phase before T_j completes its **Read** phase

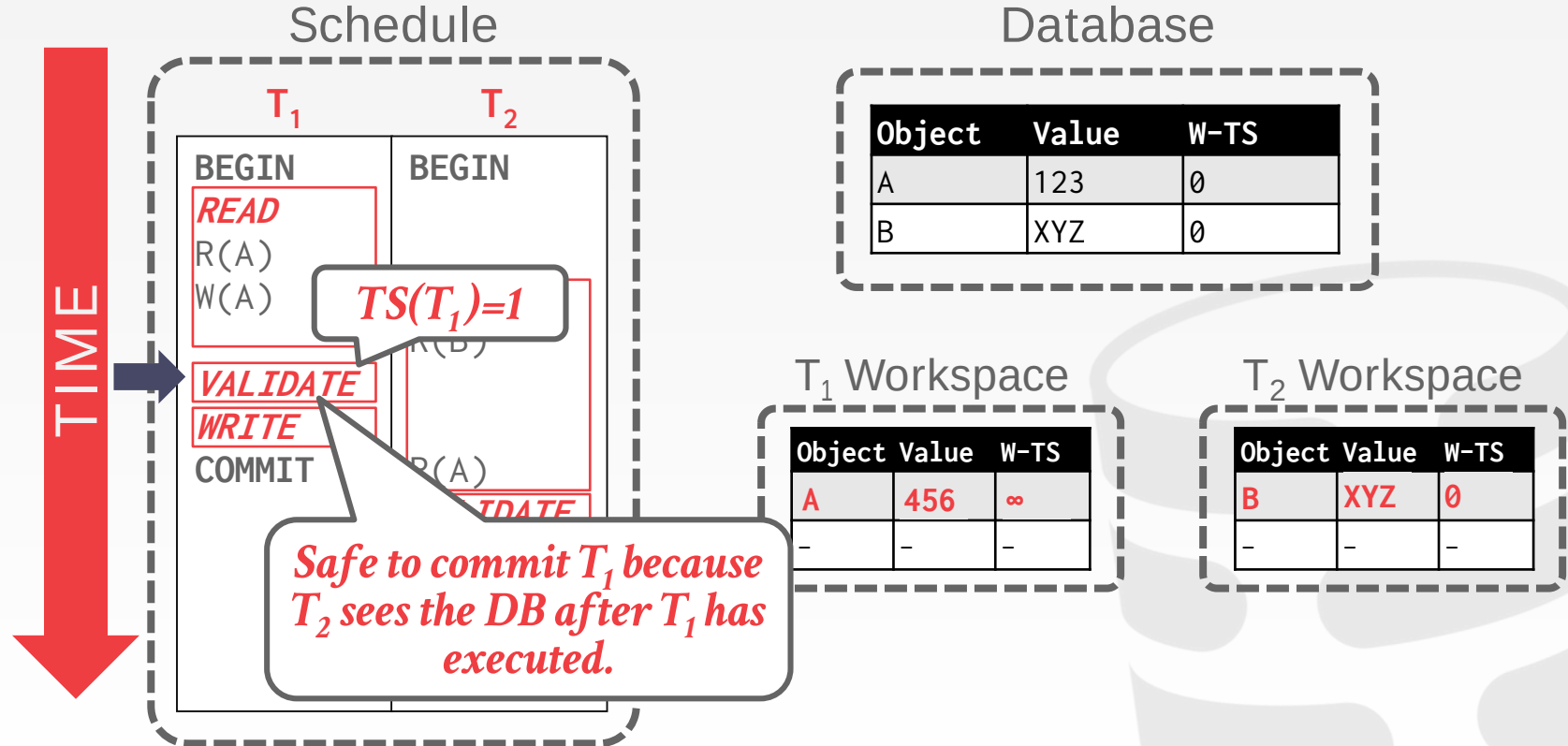
And T_i does not write to any object that is either read or written by T_j :

- $\text{WriteSet}(T_i) \cap \text{ReadSet}(T_j) = \emptyset$
- $\text{WriteSet}(T_i) \cap \text{WriteSet}(T_j) = \emptyset$

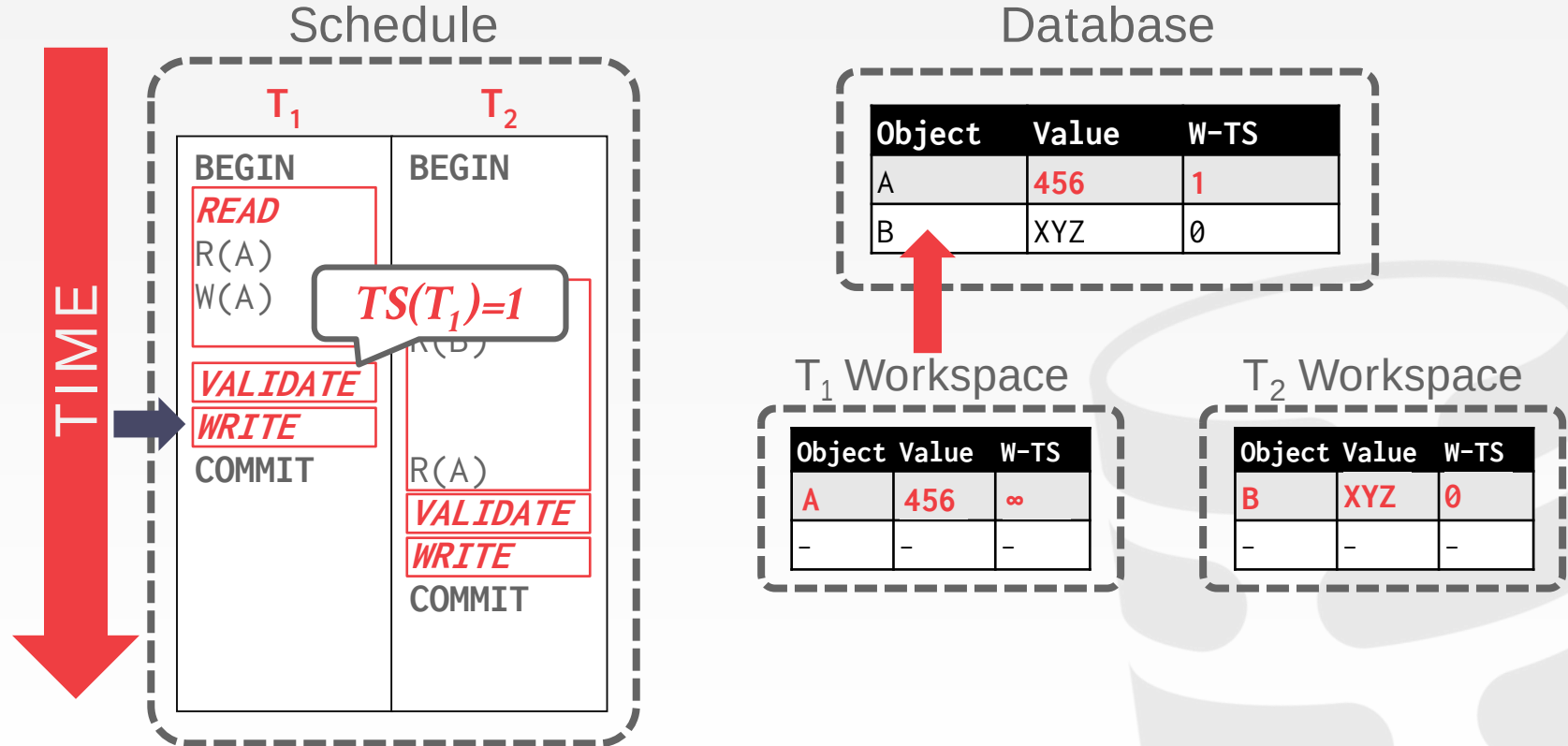
OCC – VALIDATION STEP #3



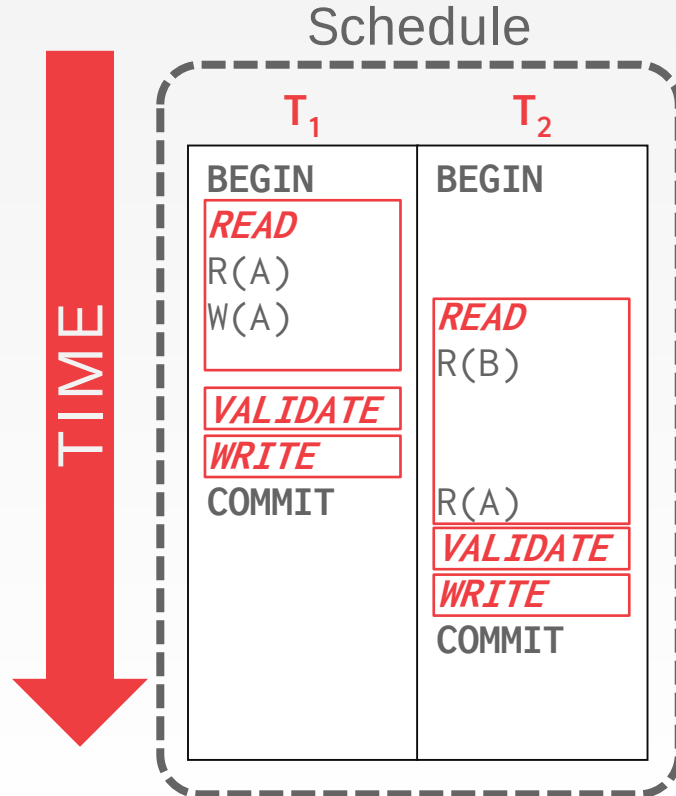
OCC – VALIDATION STEP #3



OCC – VALIDATION STEP #3



OCC – VALIDATION STEP #3



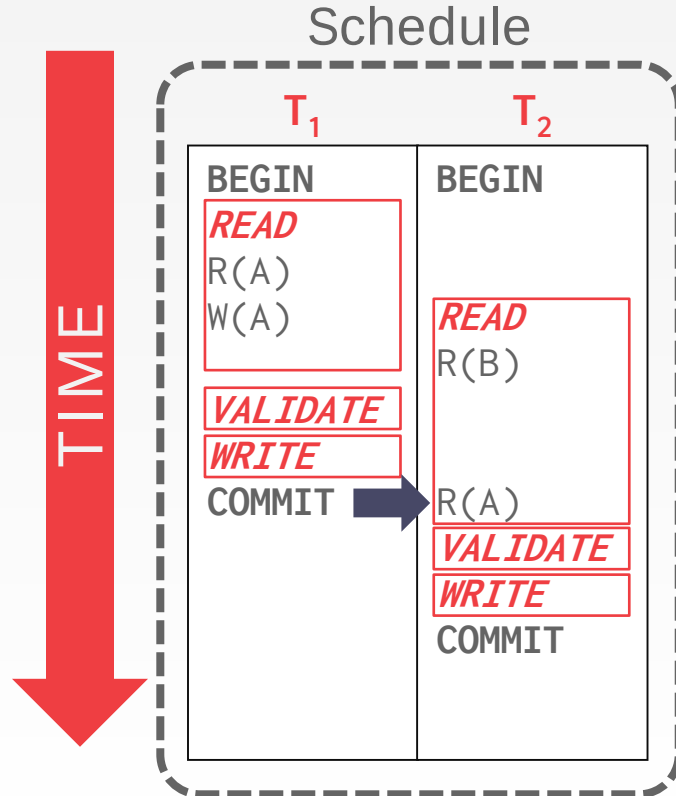
Database

Object	Value	W-TS
A	456	1
B	XYZ	0

T_2 Workspace

Object	Value	W-TS
B	XYZ	0
-	-	-

OCC – VALIDATION STEP #3



Database

Object	Value	W-TS
A	456	1
B	XYZ	0

T₂ Workspace

Object	Value	W-TS
B	XYZ	0
A	456	1

OCC – OBSERVATIONS

OCC works well when the # of conflicts is low:

- All txns are read-only (ideal).
- Txns access disjoint subsets of data.

If the database is large and the workload is not skewed, then there is a low probability of conflict, so again locking is wasteful.

OCC – PERFORMANCE ISSUES

High overhead for copying data locally.

Validation/Write phase bottlenecks.

Aborts are more wasteful than in 2PL because they only occur after a txn has already executed.

OBSERVATION

When a txn commits, all previous T/O schemes check to see whether there is a conflict with concurrent txns.

→ This requires latches.

If you have a lot of concurrent txns, then this is slow even if the conflict rate is low.

PARTITION-BASED T/O

Split the database up in disjoint subsets called *horizontal partitions* (aka shards).

Use timestamps to order txns for serial execution at each partition.

→ Only check for conflicts between txns that are running in the same partition.

DATABASE PARTITIONING

```
CREATE TABLE customer (  
  c_id INT PRIMARY KEY,  
  c_email VARCHAR UNIQUE,  
  :  
);
```

```
CREATE TABLE orders (  
  o_id INT PRIMARY KEY,  
  o_c_id INT REFERENCES  
    ↳ customer (c_id),  
  :  
);
```

```
CREATE TABLE oitems (  
  oi_id INT PRIMARY KEY,  
  oi_o_id INT REFERENCES  
    ↳ orders (o_id),  
  oi_c_id INT REFERENCES  
    ↳ orders (o_c_id),  
  :  
);
```

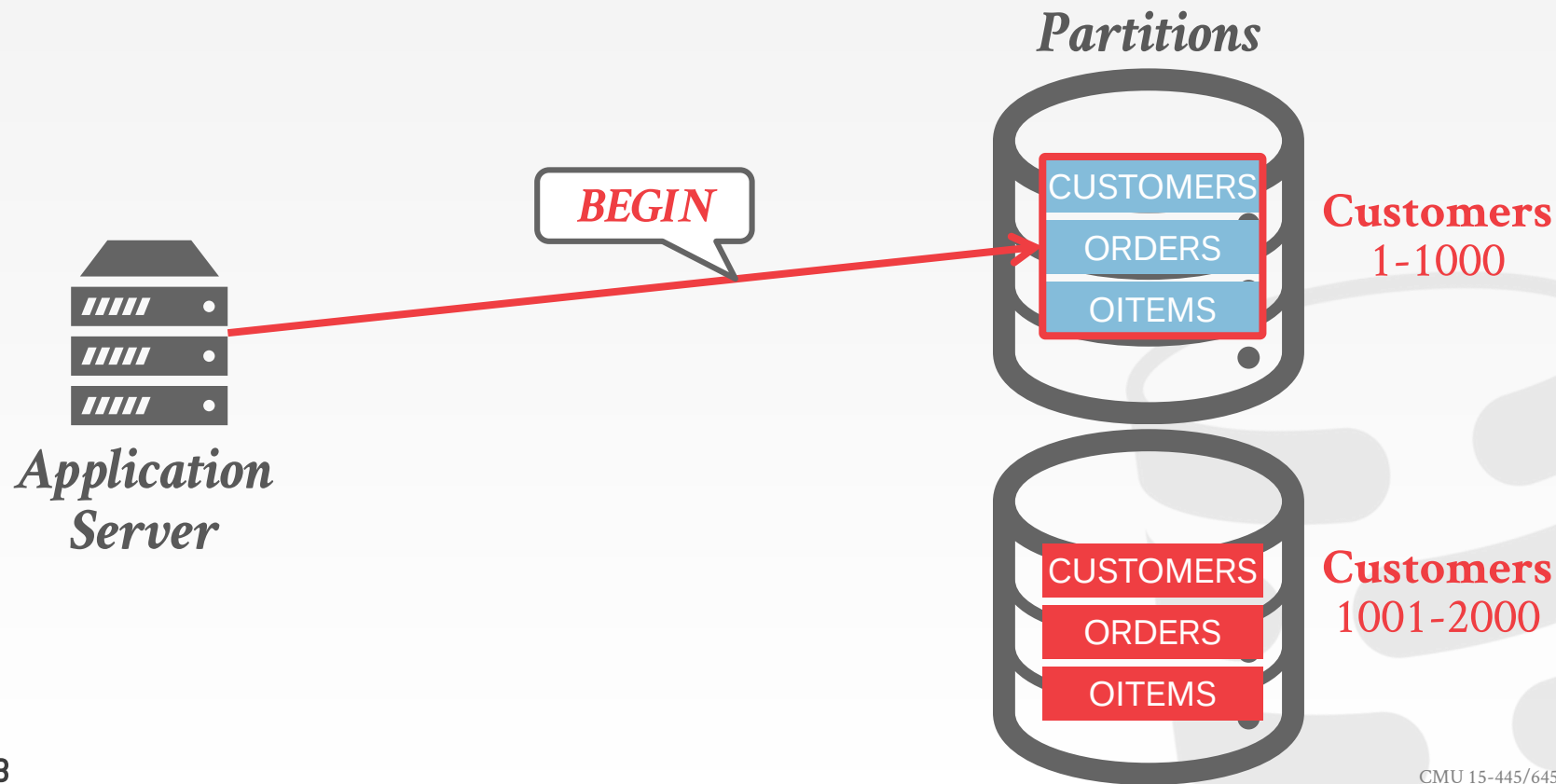
DATABASE PARTITIONING

```
CREATE TABLE customer (  
  c_id INT PRIMARY KEY,  
  c_email VARCHAR UNIQUE,  
  :  
);
```

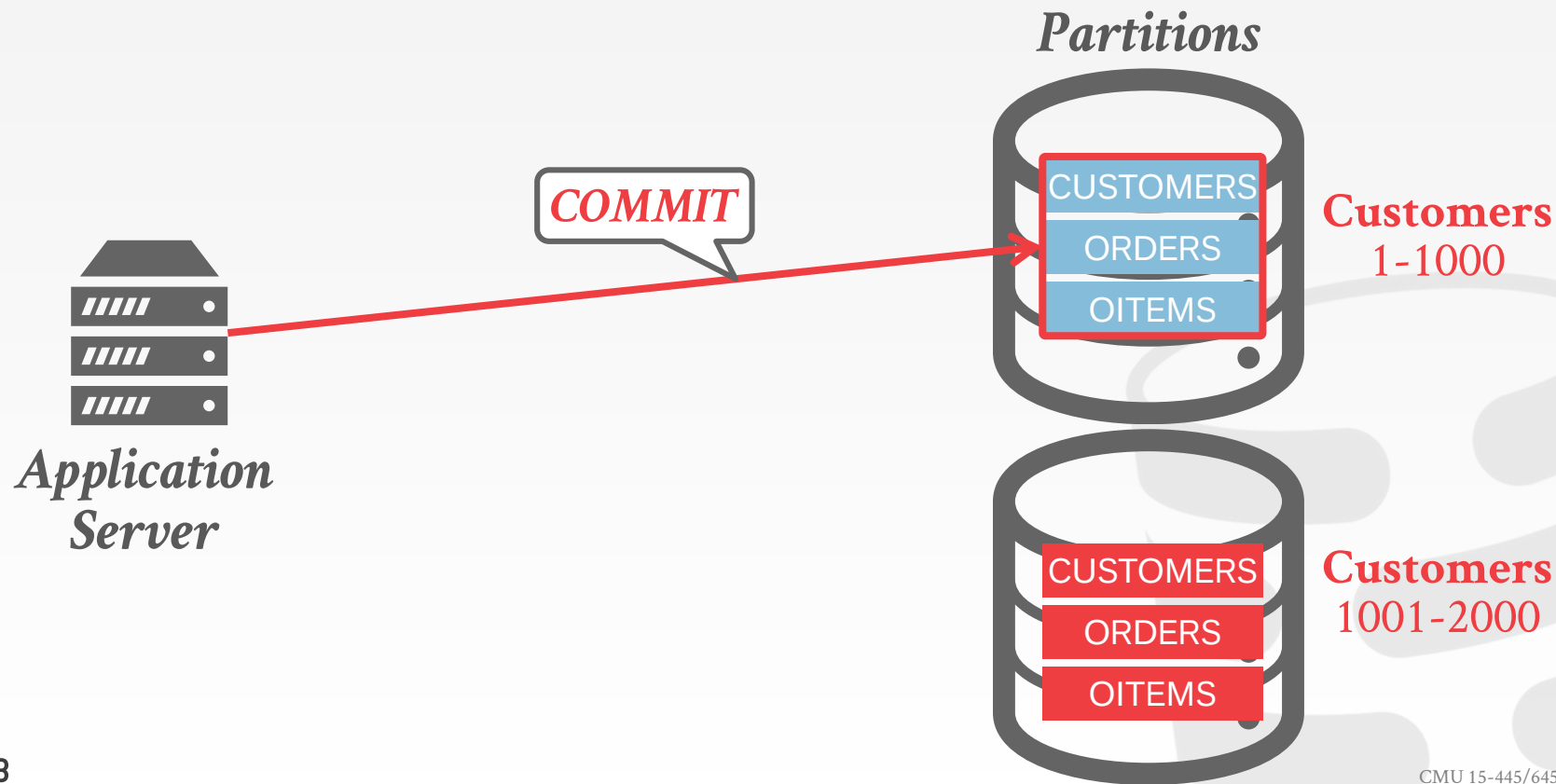
```
CREATE TABLE orders (  
  o_id INT PRIMARY KEY,  
  o_c_id INT REFERENCES  
    ↳ customer (c_id),  
  :  
);
```

```
CREATE TABLE oitems (  
  oi_id INT PRIMARY KEY,  
  oi_o_id INT REFERENCES  
    ↳ orders (o_id),  
  oi_c_id INT REFERENCES  
    ↳ orders (o_c_id),  
  :  
);
```

HORIZONTAL PARTITIONING



HORIZONTAL PARTITIONING



PARTITION-BASED T/O

Txns are assigned timestamps based on when they arrive at the DBMS.

Partitions are protected by a single lock:

- Each txn is queued at the partitions it needs.
- The txn acquires a partition's lock if it has the lowest timestamp in that partition's queue.
- The txn starts when it has all of the locks for all the partitions that it will read/write.



PARTITION-BASED T/O – READS

Txns can read anything that they want at the partitions that they have locked.

If a txn tries to access a partition that it does not have the lock, it is aborted + restarted.



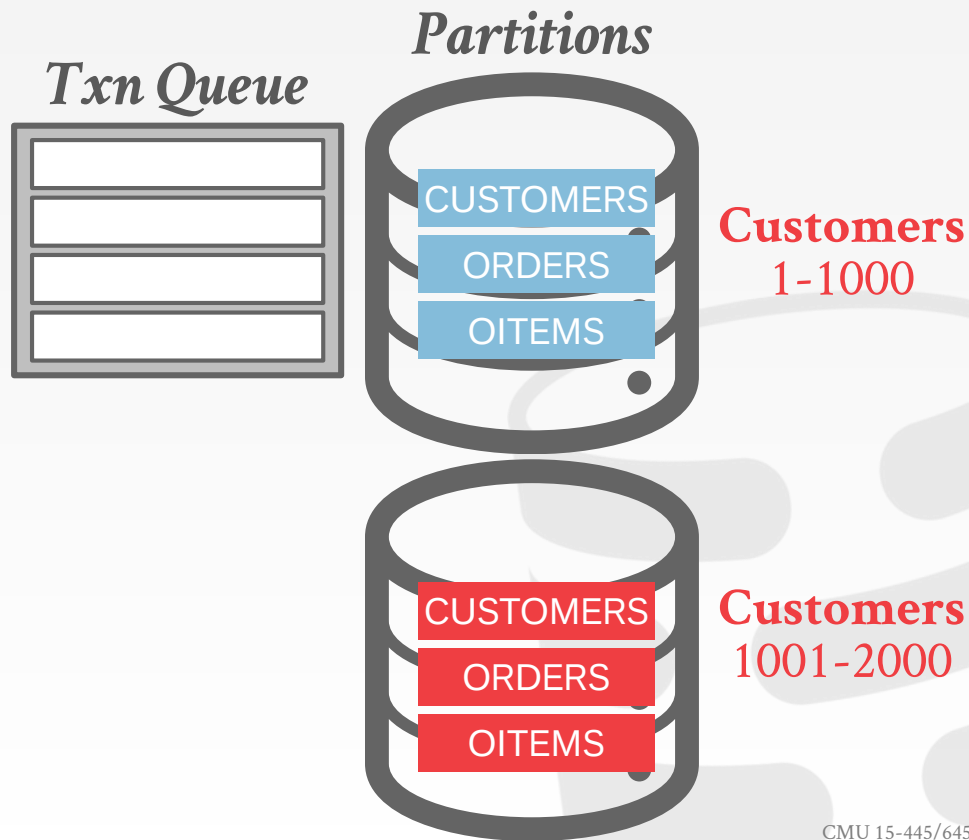
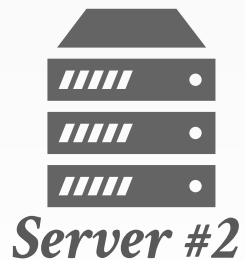
PARTITION-BASED T/O – WRITES

All updates occur in place.

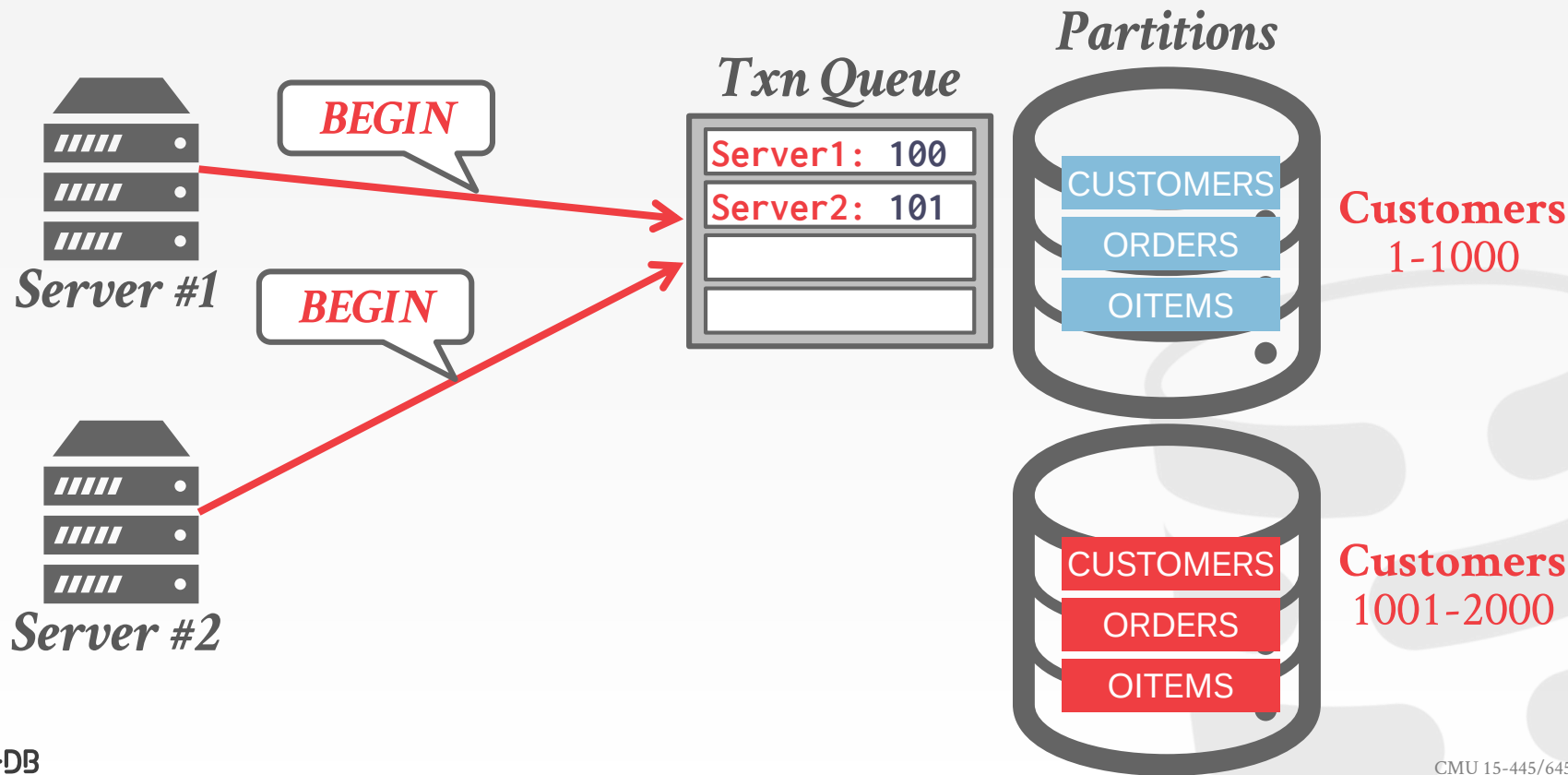
→ Maintain a separate in-memory buffer to undo changes if the txn aborts.

If a txn tries to write to a partition that it does not have the lock, it is aborted + restarted.

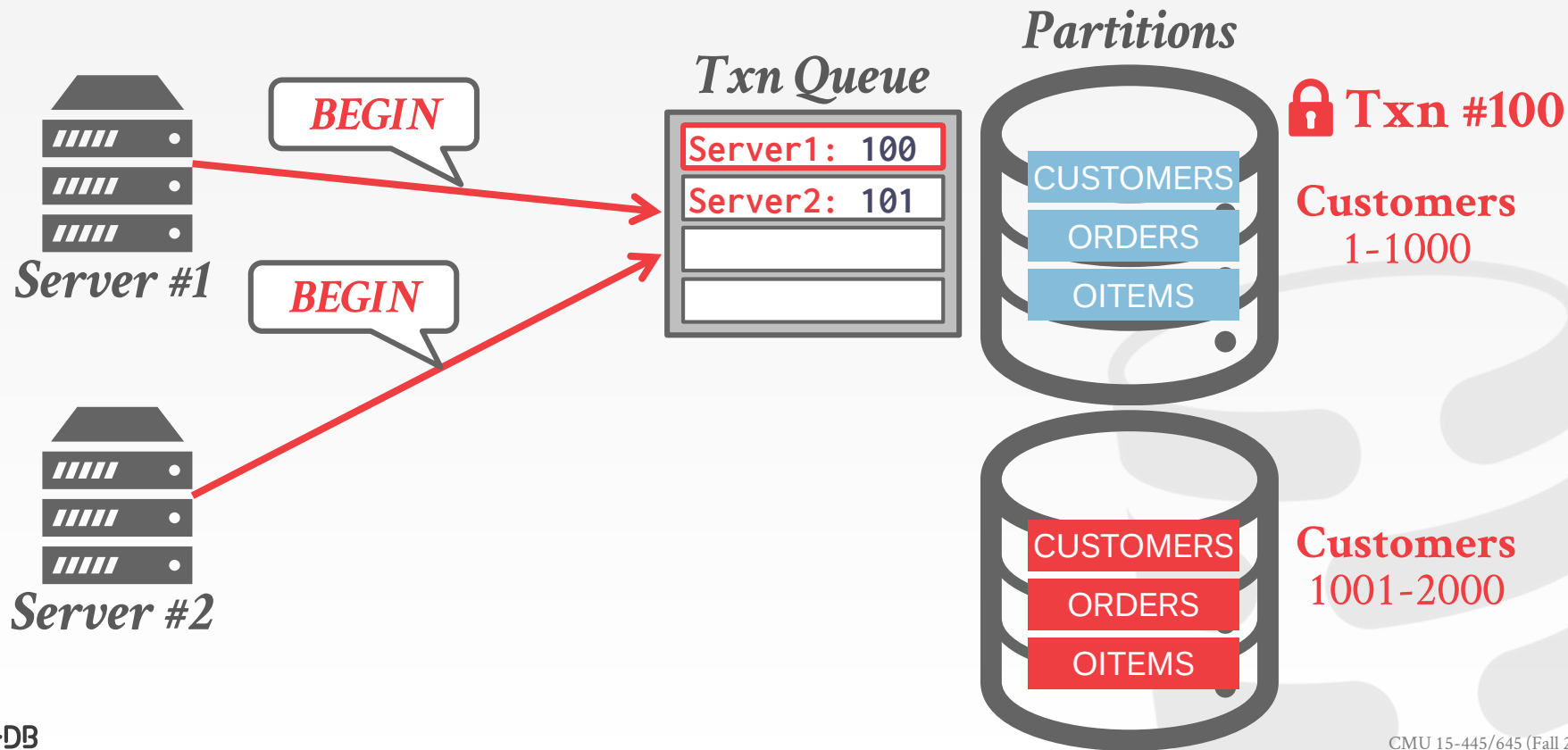
PARTITION-BASED T/O



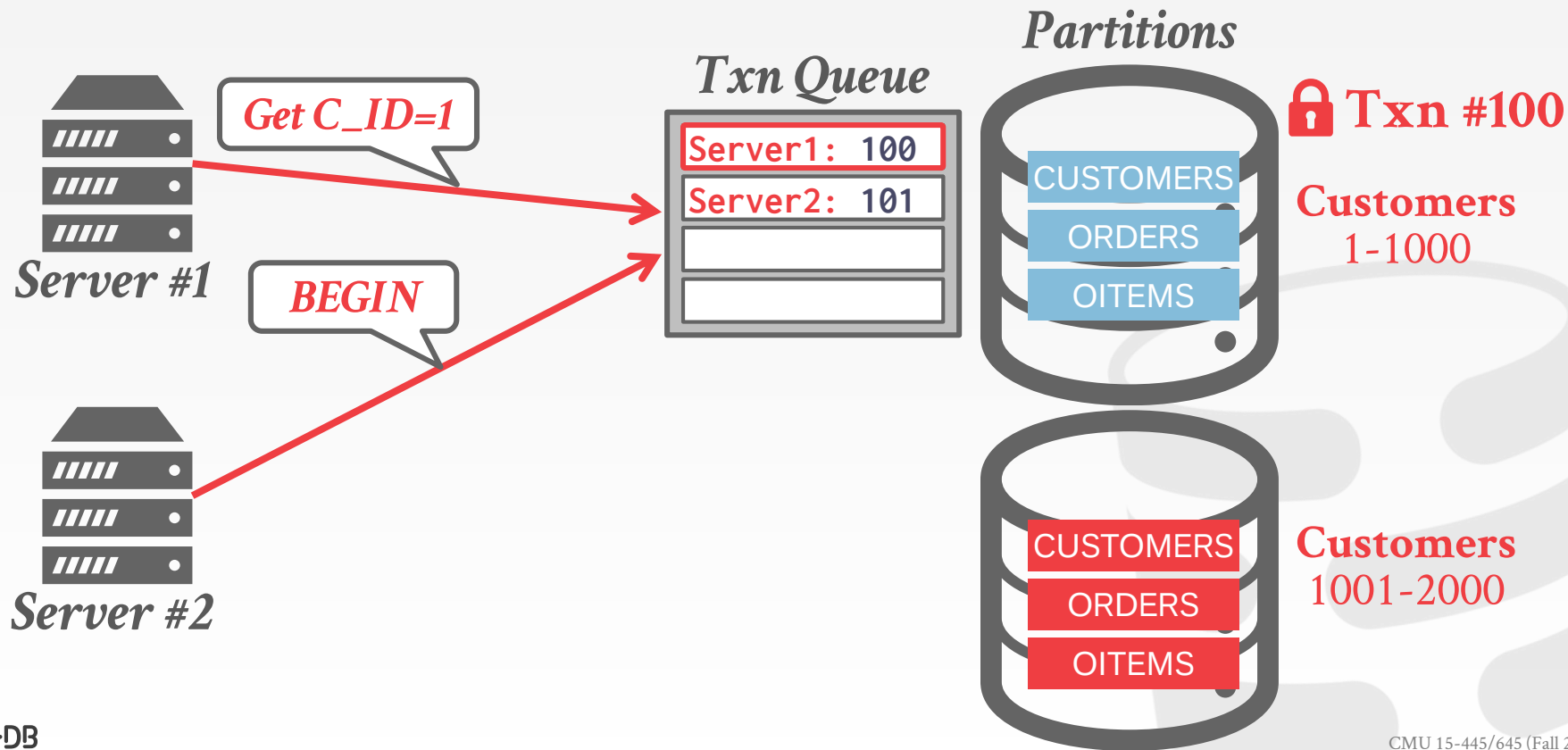
PARTITION-BASED T/O



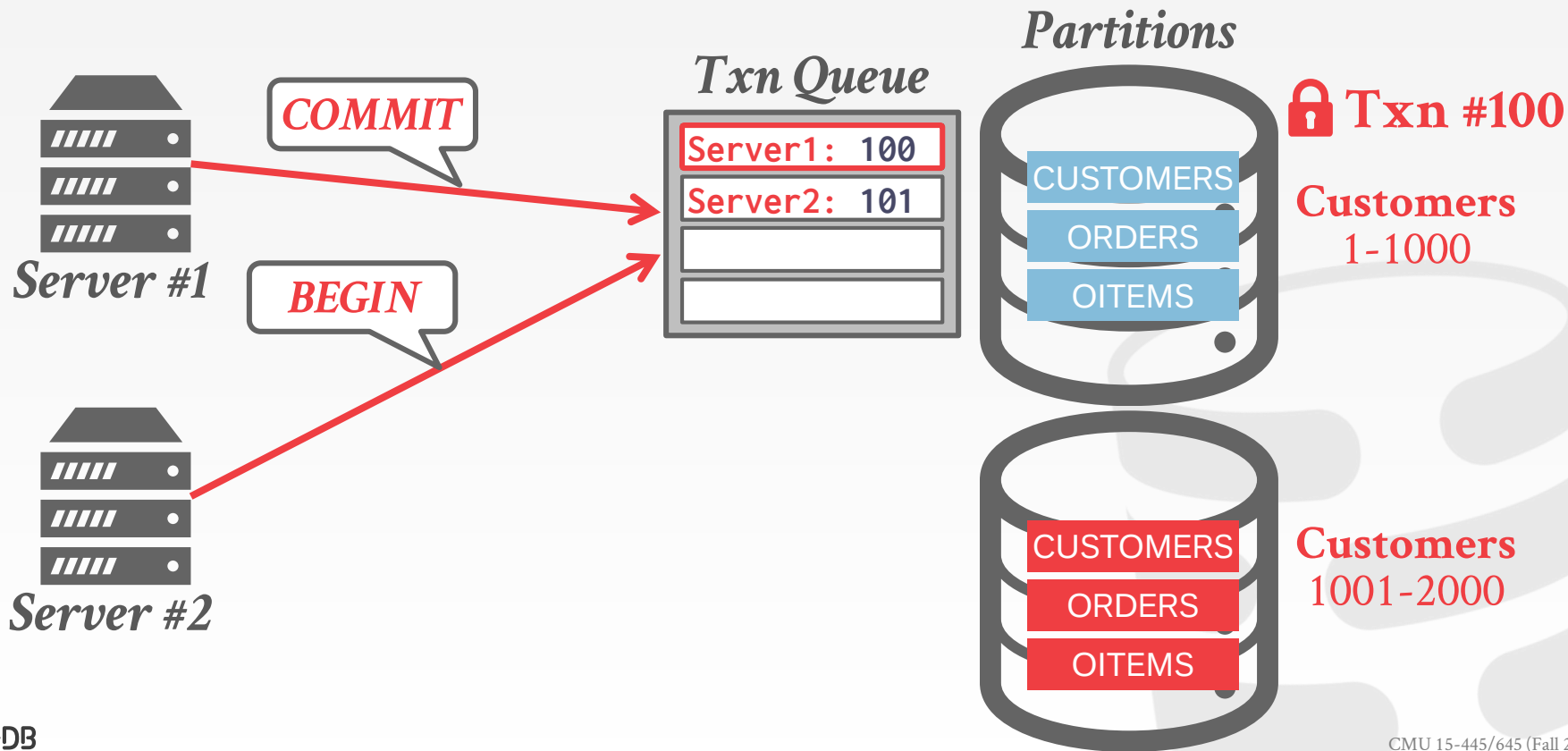
PARTITION-BASED T/O



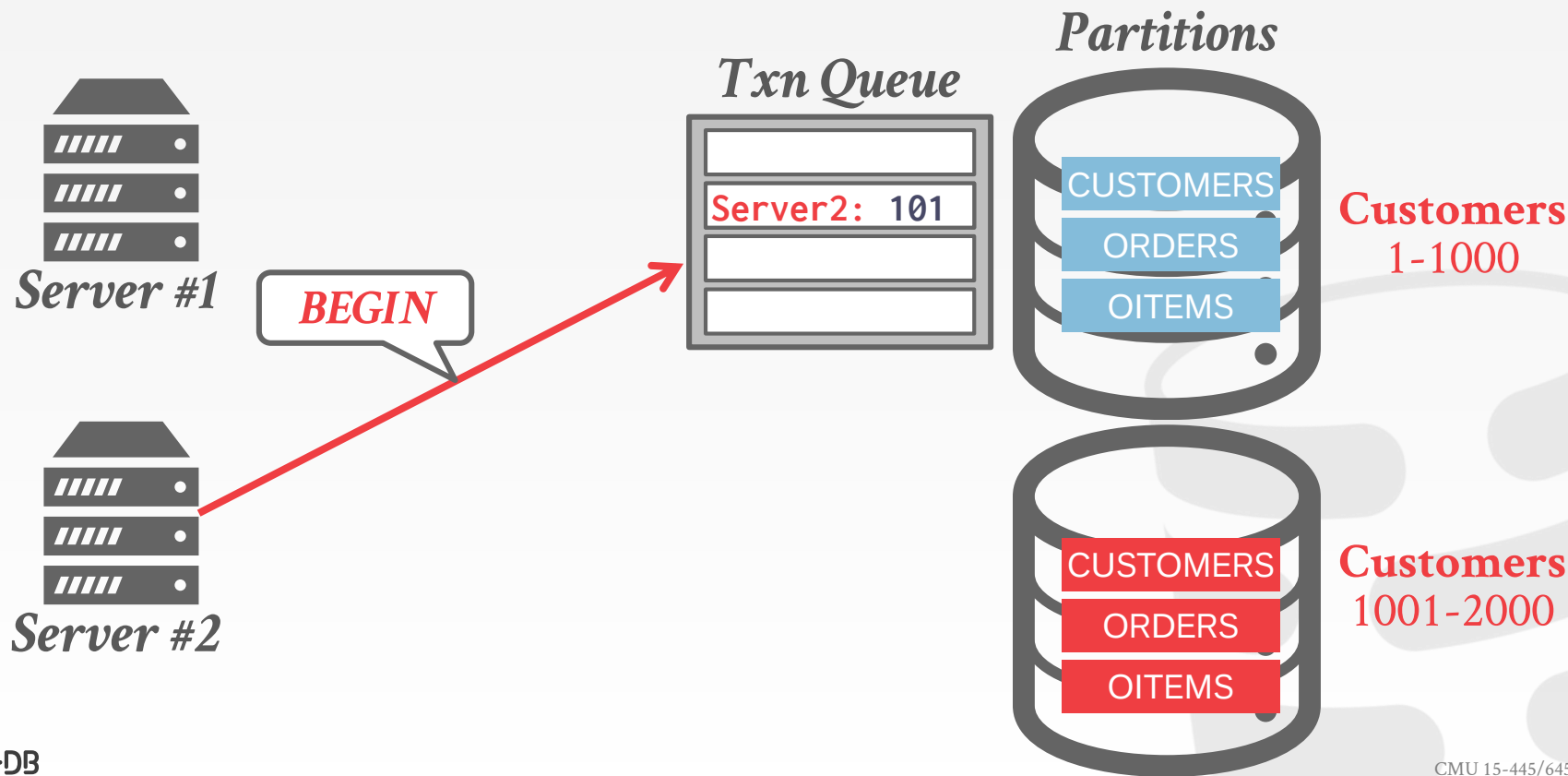
PARTITION-BASED T/O



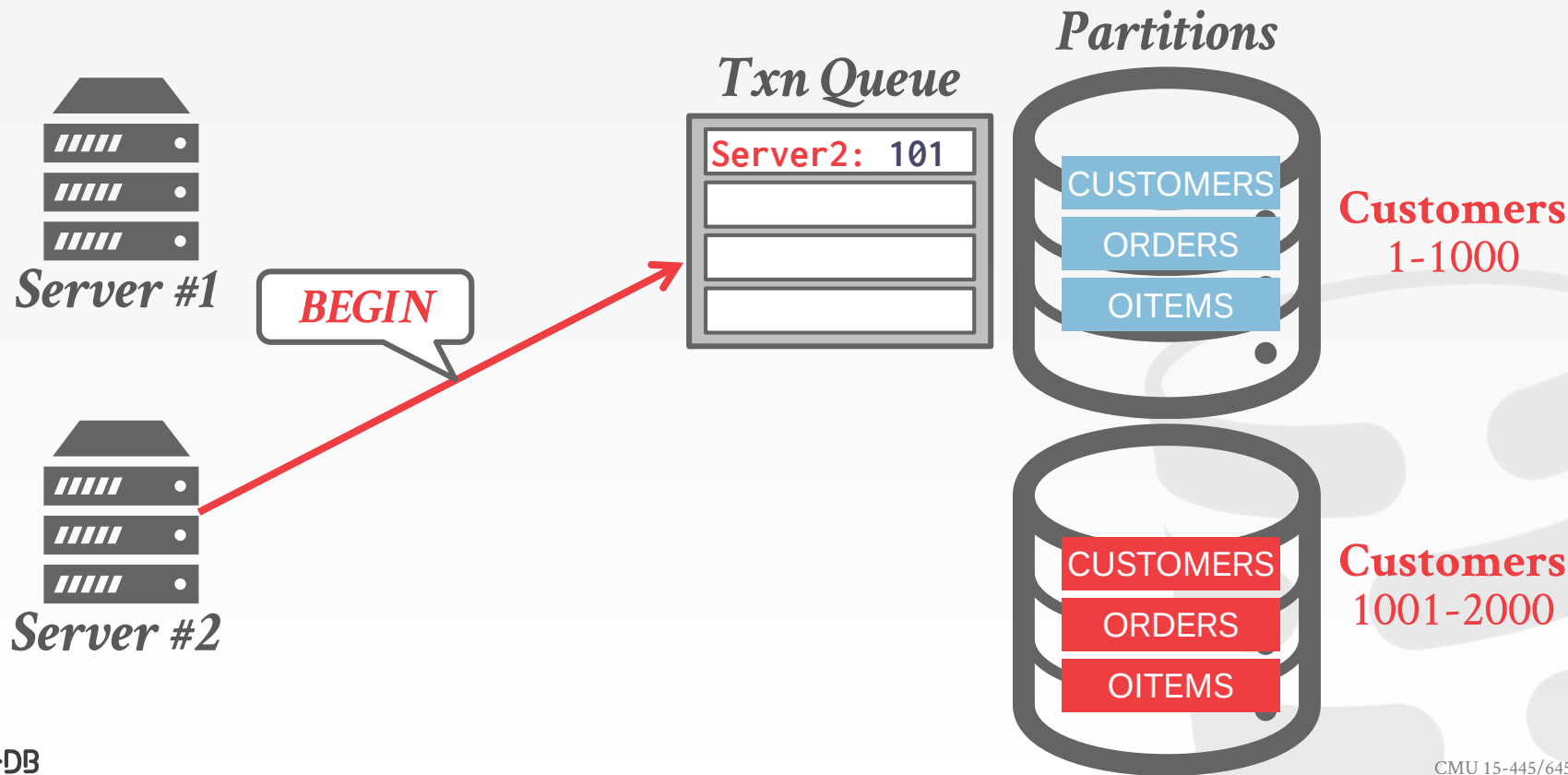
PARTITION-BASED T/O



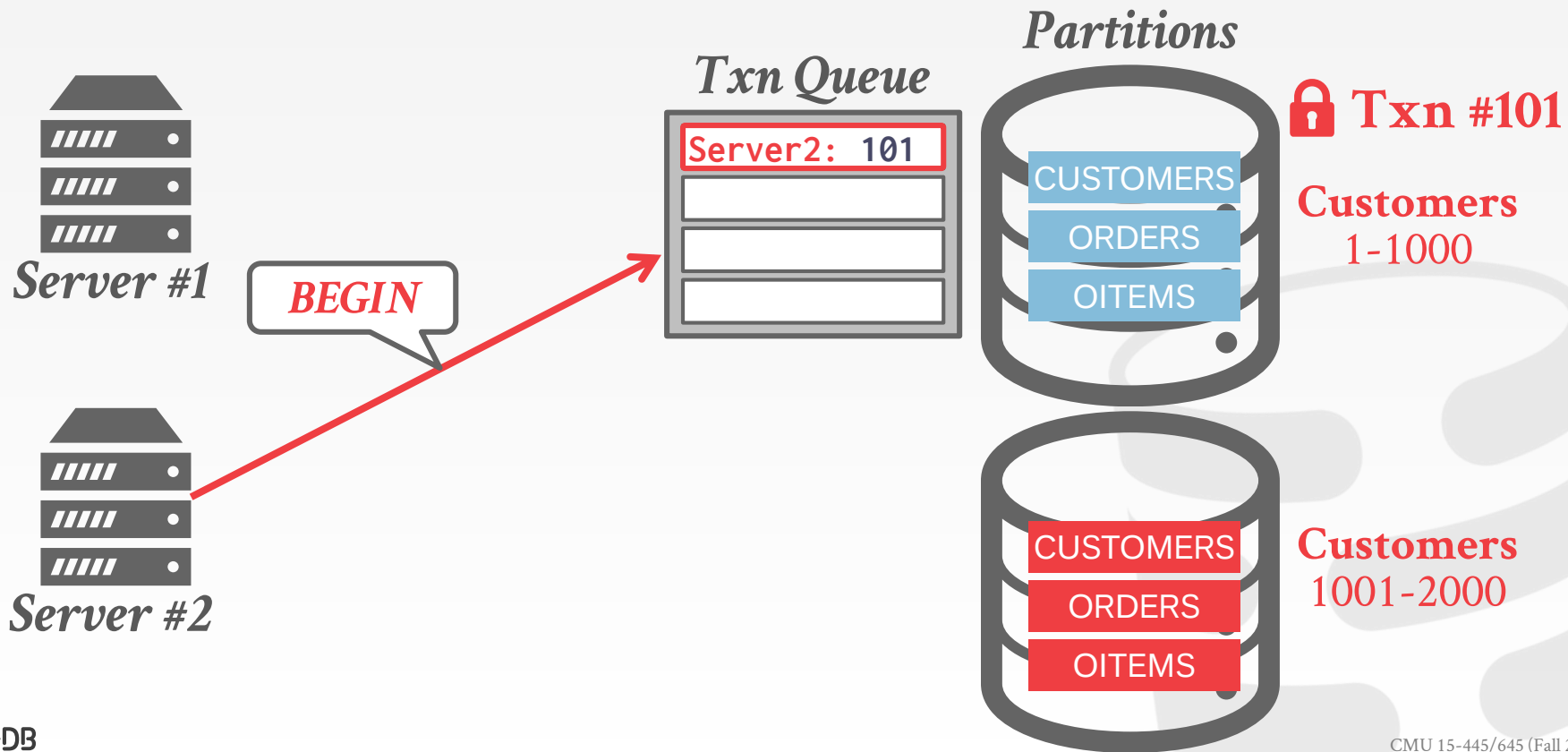
PARTITION-BASED T/O



PARTITION-BASED T/O



PARTITION-BASED T/O



PARTITIONED T/O – PERFORMANCE ISSUES

Partition-based T/O protocol is fast if:

- The DBMS knows what partitions the txn needs before it starts.
- Most (if not all) txns only need to access a single partition.

Multi-partition txns causes partitions to be idle while txn executes.

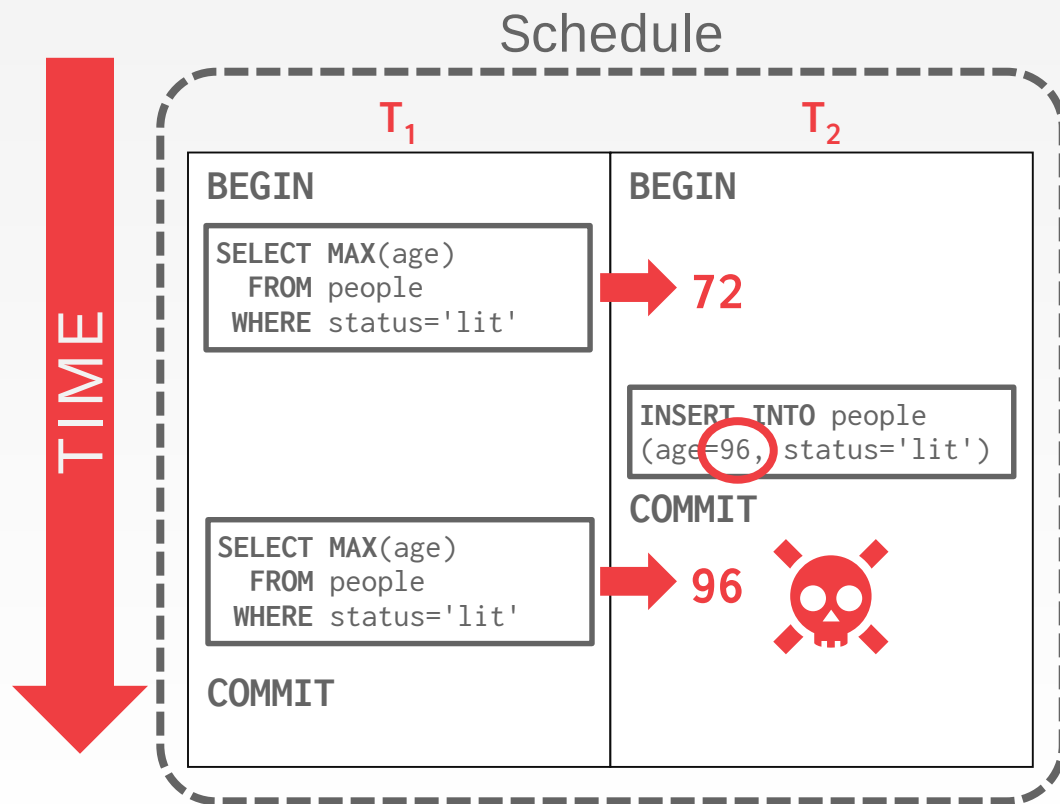
DYNAMIC DATABASES

Recall that so far we have only dealing with transactions that read and update data.

But now if we have insertions, updates, and deletions, we have new problems...



THE PHANTOM PROBLEM



```
CREATE TABLE people (
  id SERIAL,
  name VARCHAR,
  age INT,
  status VARCHAR
);
```

WTF?

How did this happen?

→ Because T_1 locked only existing records and not ones under way!

Conflict serializability on reads and writes of individual items guarantees serializability only if the set of objects is fixed.

PREDICATE LOCKING

Lock records that satisfy a logical predicate:

→ Example: **status='lit'**

In general, predicate locking has a lot of locking overhead.

Index locking is a special case of predicate locking that is potentially more efficient.

INDEX LOCKING

If there is a dense index on the status field then the txn can lock index page containing the data with **status='lit'**.

If there are no records with **status='lit'**, the txn must lock the index page where such a data entry would be, if it existed.

LOCKING WITHOUT AN INDEX

If there is no suitable index, then the txn must obtain:

- A lock on every page in the table to prevent a record's **status='lit'** from being changed to **lit**.
- The lock for the table itself to prevent records with **status='lit'** from being added or deleted.

REPEATING SCANS

An alternative is to just re-execute every scan again when the txn commits and check whether it gets the same result.

- Have to retain the scan set for every range query in a txn.
- Andy doesn't know of any commercial system that does this (only just Silo?).

WEAKER LEVELS OF ISOLATION

Serializability is useful because it allows programmers to ignore concurrency issues.

But enforcing it may allow too little concurrency and limit performance.

We may want to use a weaker level of consistency to improve scalability.

ISOLATION LEVELS

Controls the extent that a txn is exposed to the actions of other concurrent txns.

Provides for greater concurrency at the cost of exposing txns to uncommitted changes:

- Dirty Reads
- Unrepeatable Reads
- Phantom Reads



ISOLATION LEVELS



Isolation (High→Low)

SERIALIZABLE: No phantoms, all reads repeatable, no dirty reads.

REPEATABLE READS: Phantoms may happen.

READ COMMITTED: Phantoms and unrepeatable reads may happen.

READ UNCOMMITTED: All of them may happen.

ISOLATION LEVELS

	Dirty Read	Unrepeatable Read	Phantom
SERIALIZABLE	No	No	No
REPEATABLE READ	No	No	Maybe
READ COMMITTED	No	Maybe	Maybe
READ UNCOMMITTED	Maybe	Maybe	Maybe

ISOLATION LEVELS

SERIALIZABLE: Obtain all locks first; plus index locks, plus strict 2PL.

REPEATABLE READS: Same as above, but no index locks.

READ COMMITTED: Same as above, but **S** locks are released immediately.

READ UNCOMMITTED: Same as above, but allows dirty reads (no **S** locks).

SQL-92 ISOLATION LEVELS

You set a txn's isolation level before you execute any queries in that txn.

Not all DBMS support all isolation levels in all execution scenarios

→ Replicated Environments

The default depends on implementation...

```
SET TRANSACTION ISOLATION LEVEL  
<isolation-level>;
```

```
BEGIN TRANSACTION ISOLATION LEVEL  
<isolation-level>;
```

ISOLATION LEVELS (2013)

	Default	Maximum
Action Ingres 10.0/10S	SERIALIZABLE	SERIALIZABLE
Aerospike	READ COMMITTED	READ COMMITTED
Greenplum 4.1	READ COMMITTED	SERIALIZABLE
MySQL 5.6	REPEATABLE READS	SERIALIZABLE
MemSQL 1b	READ COMMITTED	READ COMMITTED
MS SQL Server 2012	READ COMMITTED	SERIALIZABLE
Oracle 11g	READ COMMITTED	SNAPSHOT ISOLATION
Postgres 9.2.2	READ COMMITTED	SERIALIZABLE
SAP HANA	READ COMMITTED	SERIALIZABLE
ScaleDB 1.02	READ COMMITTED	READ COMMITTED
VoltDB	SERIALIZABLE	SERIALIZABLE

Source: [Peter Bailis](#)

SQL-92 ACCESS MODES

You can provide hints to the DBMS about whether a txn will modify the database during its lifetime.

Only two possible modes:

- **READ WRITE** (Default)
- **READ ONLY**

Not all DBMSs will optimize execution if you set a txn to in **READ ONLY** mode.

```
SET TRANSACTION <access-mode>;
```

```
BEGIN TRANSACTION <access-mode>;
```

CONCLUSION

Every concurrency control can be broken down into the basic concepts that I've described in the last two lectures.

I'm not showing benchmark results because I don't want you to get the wrong idea.

NEXT CLASS

Multi-Version Concurrency Control



CMU 14-455 (2019) · 课程资料包 @ShowMeAI



视频

中英双语字幕



课件

一键打包下载



笔记

官方笔记翻译



代码

作业项目解析



视频 · B 站 [扫码或点击链接]

<https://www.bilibili.com/video/BV1qf4y1J7mX>



课件 & 代码 · 博客 [扫码或点击链接]

<http://blog.showmeai.tech/cmu-14-455/>

高级 SQL

数据库

聚合

内存管理

树索引

排序

查询规划

索引构建

分组

并发控制

查询执行

Awesome AI Courses Notes Cheatsheets 是 [ShowMeAI](#) 资料库的分支系列，覆盖最具知名度的 **TOP50+** 门 AI 课程，旨在为读者和学习者提供一整套高品质中文学习笔记和速查表。

点击课程名称，跳转至课程**资料包**页面，**一键下载**课程全部资料！

机器学习	深度学习	自然语言处理	计算机视觉
Stanford · CS229	Stanford · CS230	Stanford · CS224n	Stanford · CS231n
# Awesome AI Courses Notes Cheatsheets · 持续更新中			
知识图谱	图机器学习	深度强化学习	自动驾驶
Stanford · CS520	Stanford · CS224W	UCBerkeley · CS285	MIT · 6.S094



微信公众号

资料下载方式 2: 扫码点击**底部菜单栏**

称为 **AI 内容创作者?** 回复 [添砖加瓦]